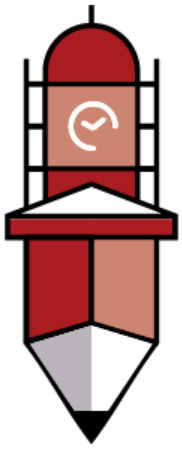


The University of Faisalabad

Lab Manual

Subject: **Data Mining**

By

M Zubair Khan **2023-BS-AI-006**

Semester **6th**

Supervised by **Sir Arham**

The University of Faisalabad

Bachelors of Science in Artificial Intelligence

Lab # 1

KNIME Introduction

Real-World Problem

- A retail company has sales transaction data but needs analysis to gain business insights.
- Goals:
  - Measure total revenue
  - Calculate average customer spending
  - Identify top-performing categories
  - Find the most profitable products

Dataset

Contains transaction-level sales records:

- Transaction ID
- Product Name
- Product Category
- Quantity
- Price per Unit
- Order Date
- Revenue (Total Amount)

Software Environment: KNIME Analytics Platform

- A visual, node-based data analytics tool.
- Uses connected nodes to read, process, analyze, and visualize data.

Practical Objectives

Students will learn to:

1. Import CSV data using the **CSV Reader** node.
2. Verify and correct data types.
3. Generate descriptive statistics.
4. Create a **Revenue** column using the **Math Formula** node.
5. Calculate key retail KPIs using **GroupBy** and **Sorter** nodes.

Key Performance Indicators (KPIs)

- Total Revenue
- Average Order Value
- Highest Revenue Category
- Most Profitable Product

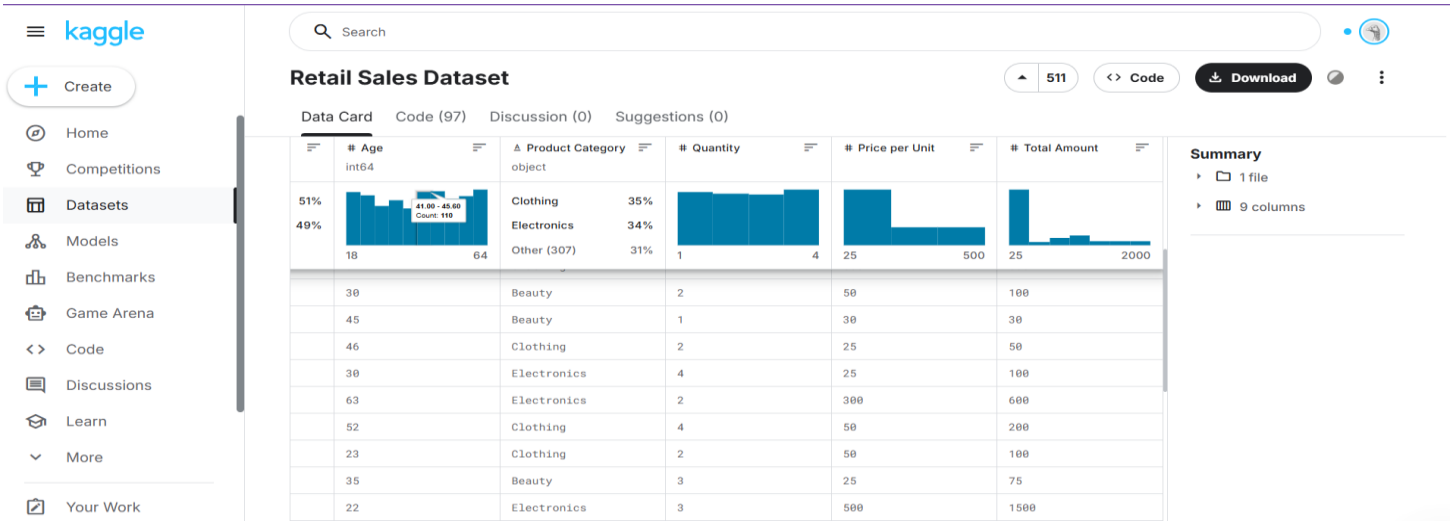
Outcome

- Transform raw sales data into meaningful business insights using a structured KNIME workflow.

Procedure

Step 1: Download the Dataset

Obtain the Retail Sales dataset in CSV format from the provided source and save it locally for analysis.

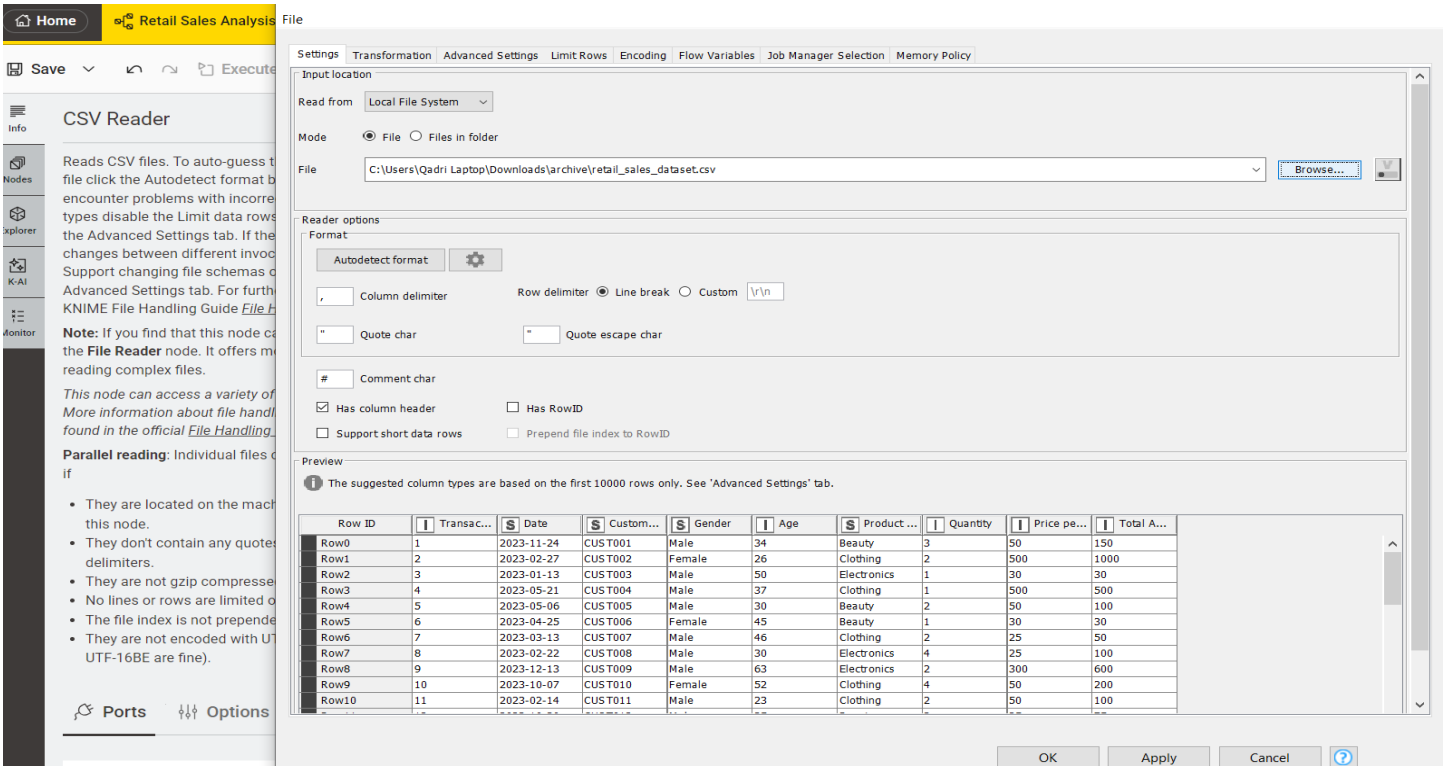


Step 2: Create a New Workflow

Launch KNIME Analytics Platform and create a new workflow named *Retail Sales Analysis* to organize the analytical process.

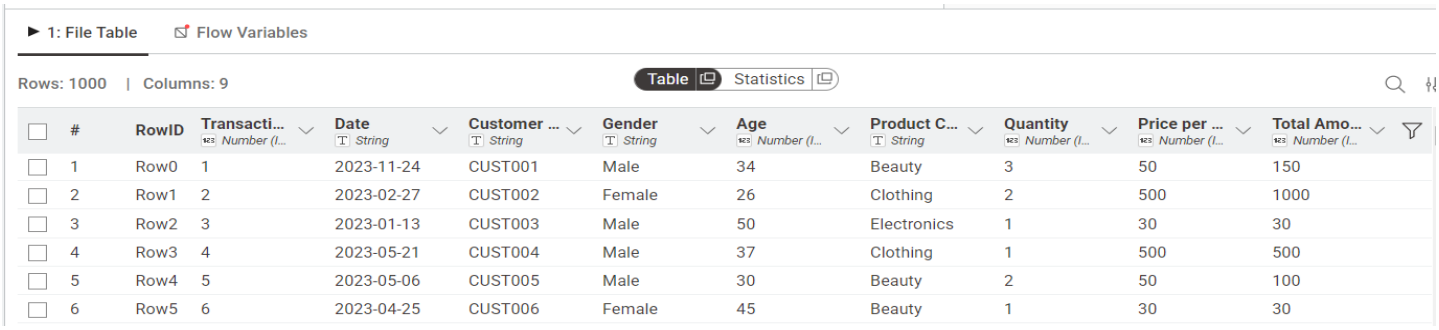
Step 3: Import the Dataset

Use the CSV Reader node to load the Retail Sales dataset into the workflow environment and verify successful execution.



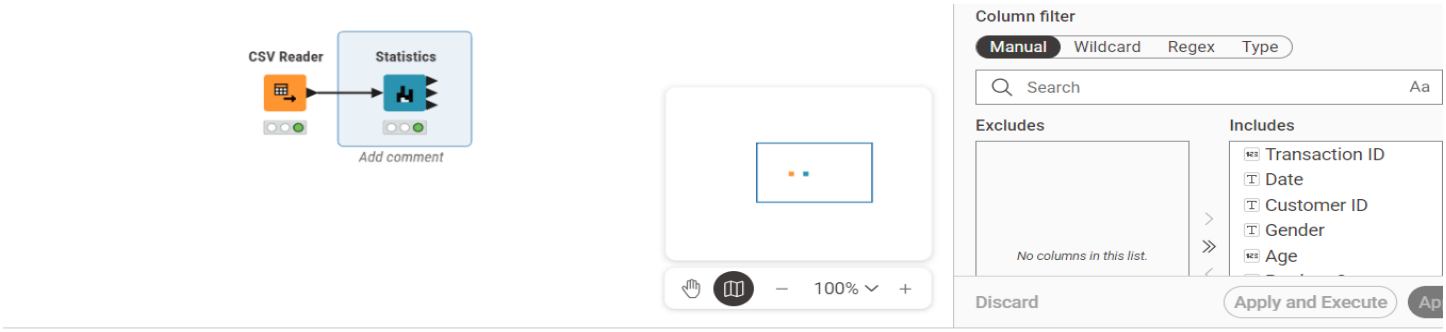
Step 4: Verify the Data Types

Inspect all columns to ensure numeric fields (Quantity, Price, Revenue) are correctly typed as Integer or Double, and categorical fields (Category, Product Name) are set as String.



Step 5: Generate Summary Statistics

Apply the Statistics node to compute descriptive measures such as mean, minimum, maximum, sum, and standard deviation for numeric attributes.



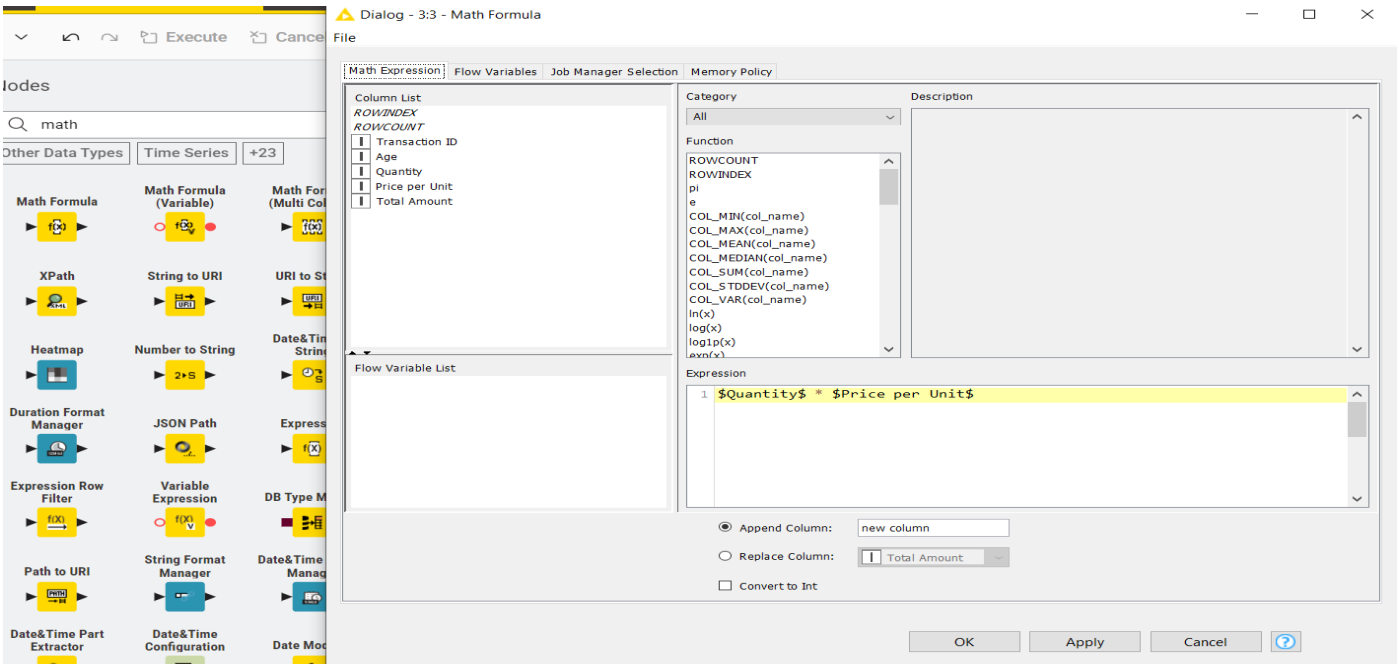
#	RowID	Column (String)	Min (Number)	Max (Number)	Mean (Number)	Std. devia... (Number)	Variance (Number)	Skewness (Number)	Kurtosis (Number)	Overall su... (Number)	No. r (Nu
1	Transa	Transaction ID	1	1,000	500.5	288.819	83,416.667	0	-1.2	500,500	0
2	Age	Age	18	64	41.392	13.681	187.182	-0.049	-1.201	41,392	0
3	Quantit	Quantity	1	4	2.514	1.133	1.283	-0.014	-1.393	2,514	0
4	Price p	Price per Unit	25	500	179.89	189.681	35,979.017	0.736	-1.139	179,890	0
5	Total A	Total Amount	25	2,000	456	559.998	313,597.347	1.376	0.815	456,000	0

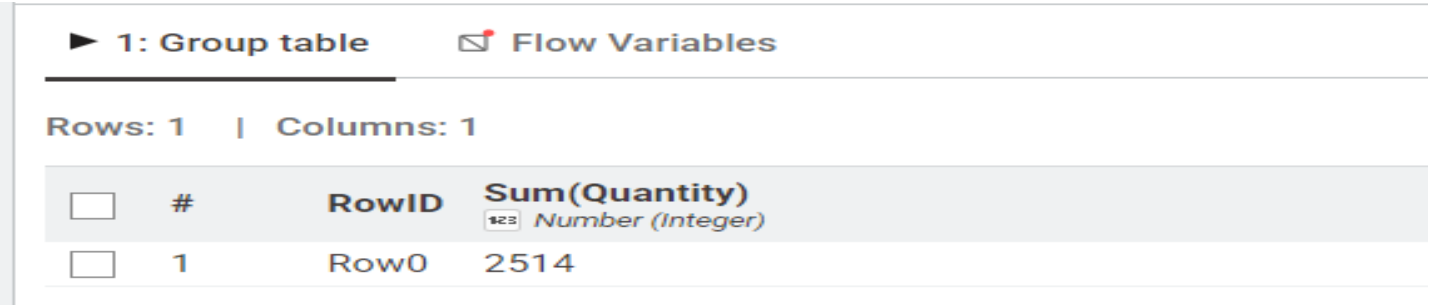
Step 6: Calculate Revenue Column

Use the Math Formula node to create a new Revenue column using the formula:

Revenue = Quantity × Price

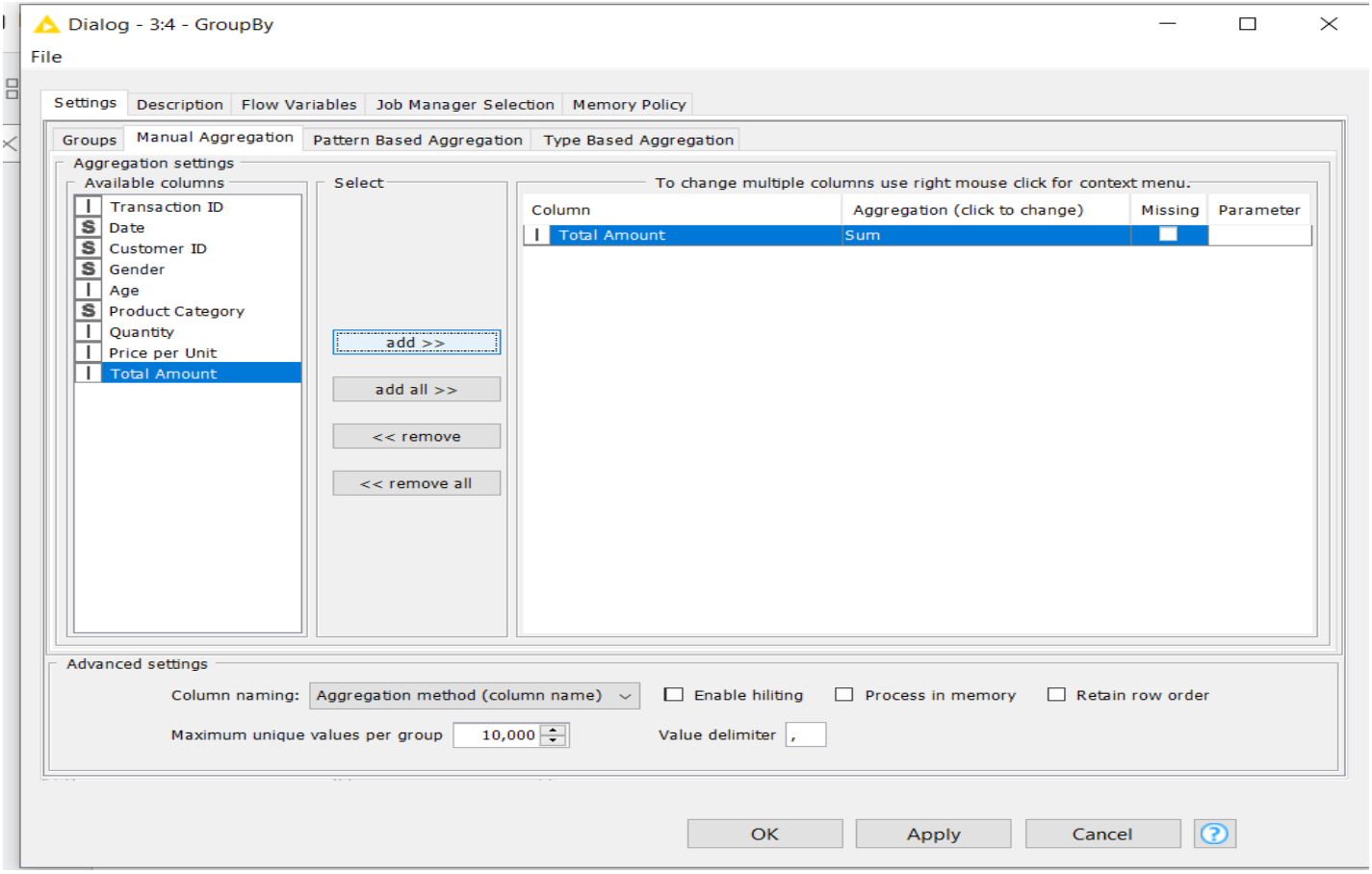
This derived column will be used for KPI calculations.





Step 7: Calculate Total Revenue (KPI 1 – Total Revenue)

Add a GroupBy node without selecting any grouping column and aggregate the Revenue column using the Sum function to compute overall sales revenue.



Step 8: View Total Revenue Result

Execute the node and observe the single output value representing total revenue across all transactions.

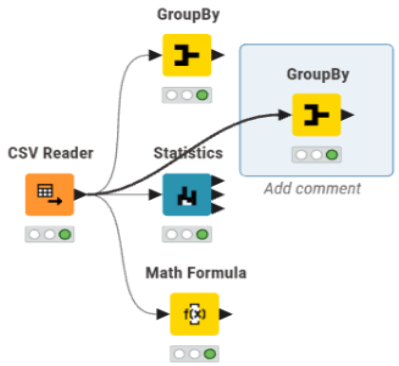
► 1: Group table Flow Variables

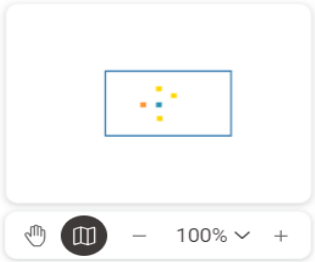
Rows: 1 | Columns: 1 Table | Statistics

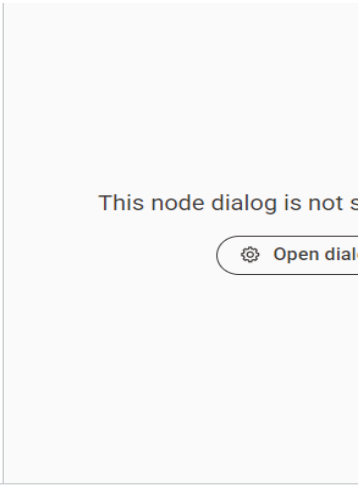
#	RowID	Sum(Total Amount) Number (Integer)
1	Row0	456000

Step 9: Calculate Revenue per Transaction

Add a GroupBy node, group by Transaction ID, and aggregate Revenue using Sum to compute total revenue for each individual order.







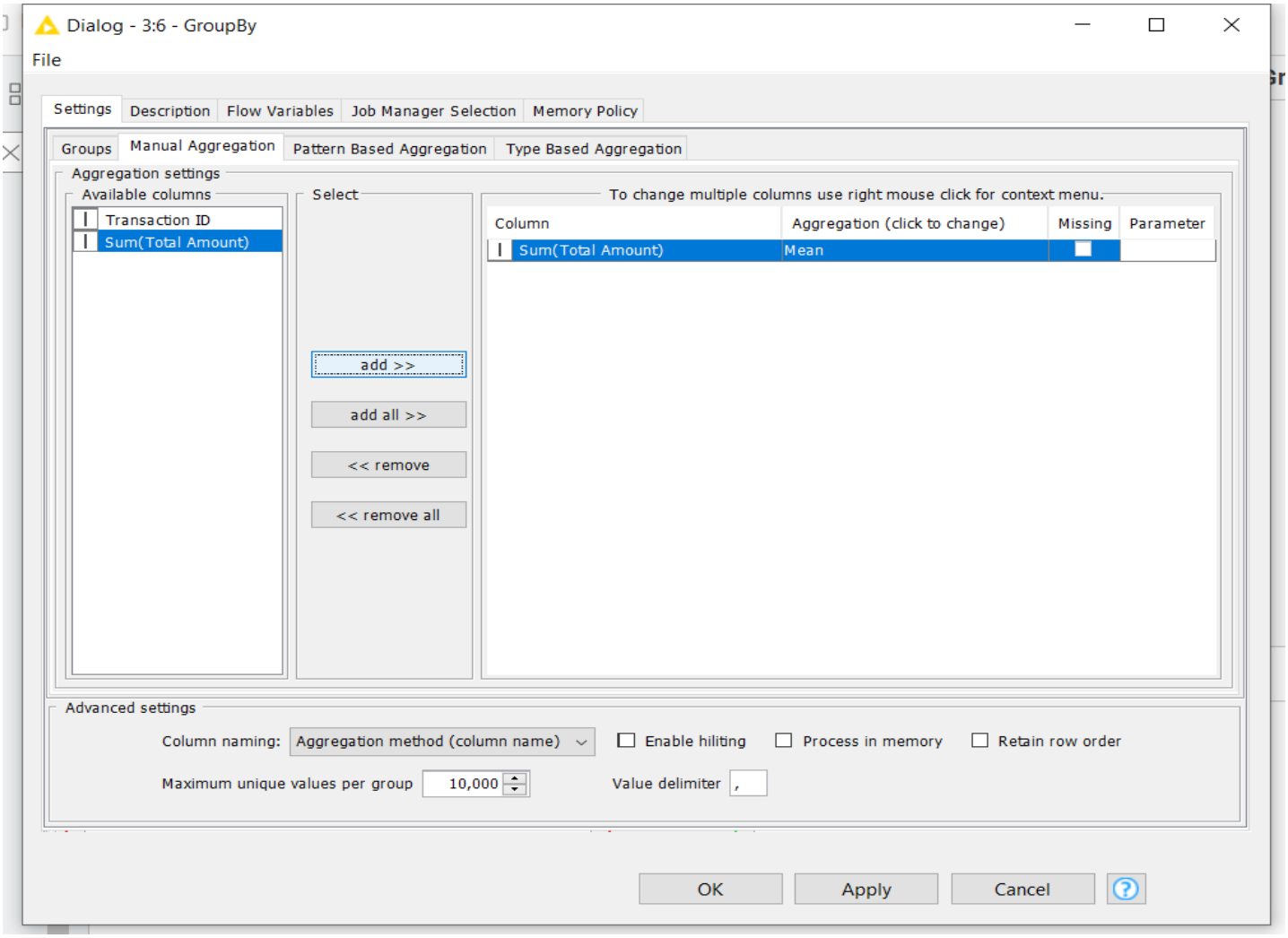
► 1: Group table Flow Variables

Rows: 1000 | Columns: 2 Table | Statistics

#	RowID	Transaction ID Number (Integer)	Sum(Total Amount) Number (Integer)
1	Row0	1	150
2	Row1	2	1000
3	Row2	3	30
4	Row3	4	500
5	Row4	5	100
6	Row5	6	30

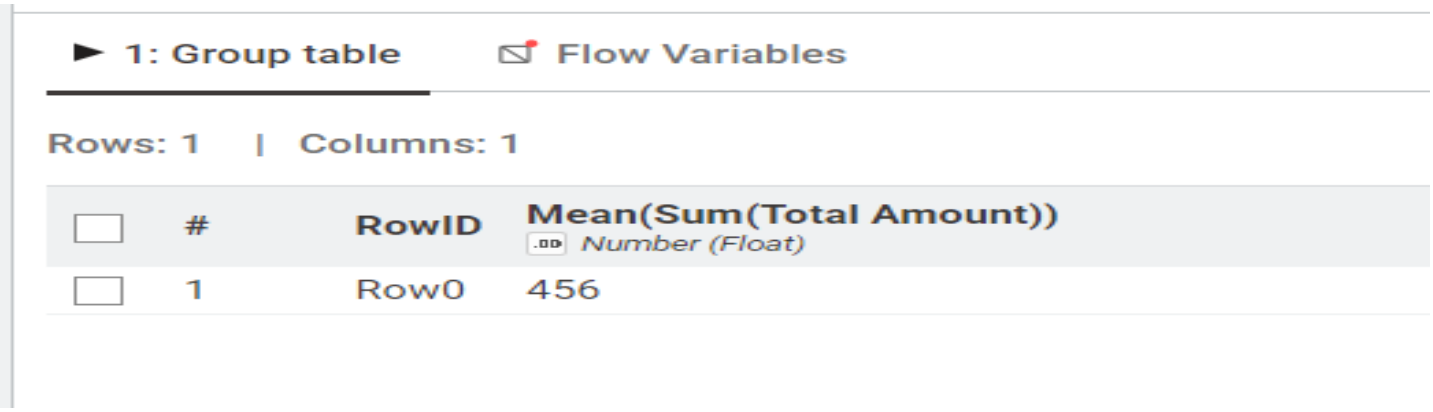
Step 10: Calculate Average of All Orders (KPI 2 – Average Order Value)

Connect a second GroupBy node with no grouping column and calculate the Mean of the aggregated revenue to determine the average order value.



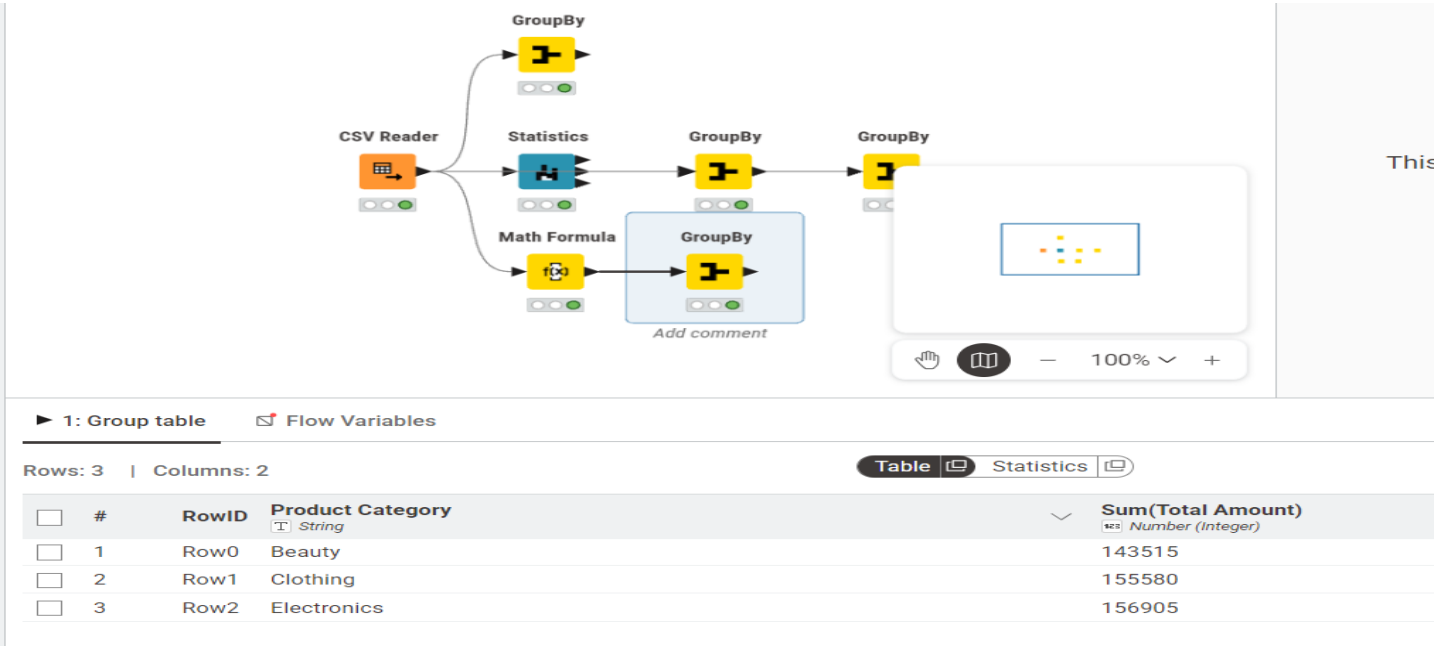
Step 11: View Average Order Value Result

Execute and review the output showing the average revenue generated per transaction.



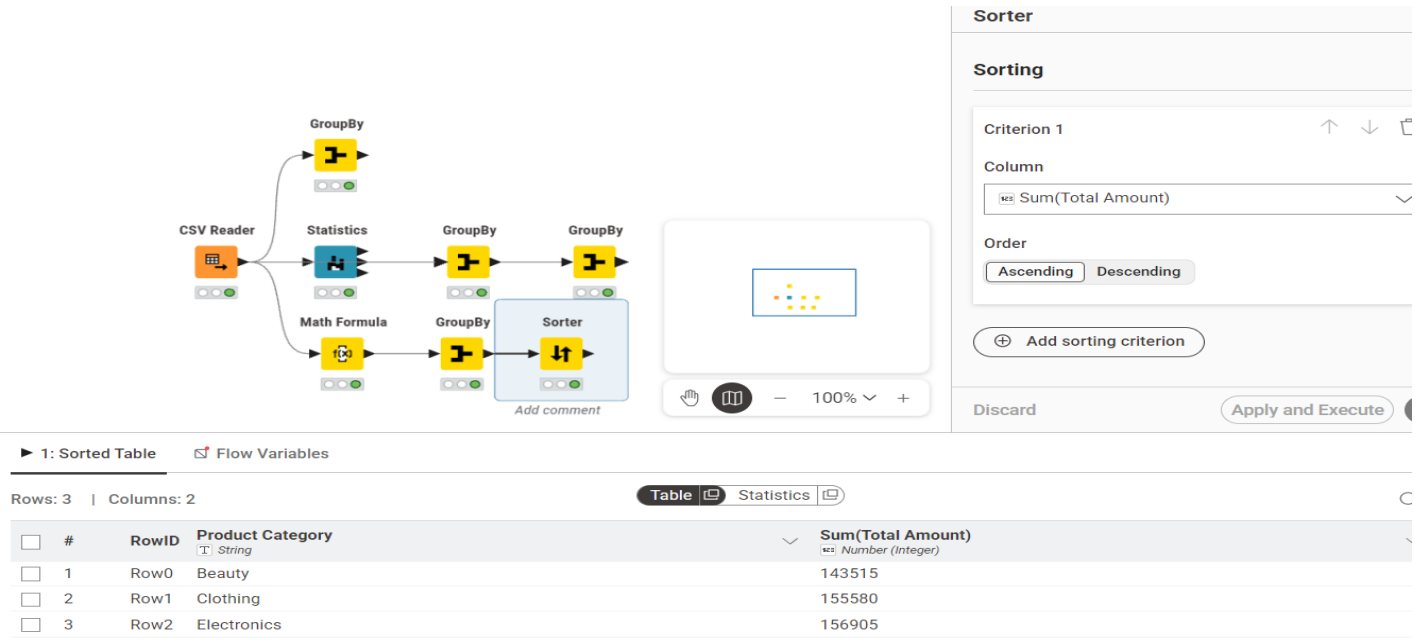
Step 12: Calculate Category Revenue (KPI 3 – Revenue by Category)

Add a GroupBy node and group by Product Category. Aggregate Revenue using Sum to compute total earnings for each category.



Step 13: Sort to Identify Highest Revenue Category

Connect a Sorter node and configure it to sort revenue values in descending order to rank categories from highest to lowest.



Outcome of the Lab

This lab demonstrates how raw transactional sales data can be systematically transformed into structured performance metrics.

The workflow converts:

Raw Data → Processed Data → Aggregated Metrics → Business Insights

Through structured node-based operations, the analysis provides measurable indicators that support revenue evaluation, category comparison, and product performance assessment.

Lab # 2

Customer Churn Prediction

Real-World Problem

- An e-commerce company wants to predict customers who may stop using its service (**churn**).
- Early prediction helps improve customer retention and reduce revenue loss.

Dataset

- **Churn_Modelling.csv**
- 10,000 customer records with 14 attributes.
- Target Variable:
  - **Exited = 1** → Churned
  - **Exited = 0** → Retained
- Important features:
  - CreditScore
  - Geography
  - Gender
  - Age
  - Tenure
  - Balance
  - NumOfProducts
  - HasCrCard
  - IsActiveMember
  - EstimatedSalary
- Removed columns:
  - RowNumber
  - CustomerId
  - Surname

Software Environment: Orange Data Mining

- Open-source visual analytics tool.

- Builds data mining workflows using drag-and-drop widgets without coding.

Practical Objectives

1. Load and inspect data using **File** and **Data Table** widgets.
2. Remove unnecessary columns and set the target variable using **Select Columns**.
3. Preprocess data (normalize and encode features).
4. Train **Logistic Regression** and **Random Forest** models.
5. Evaluate models using **10-fold Cross Validation**.
6. Analyze results with **Confusion Matrix** and **ROC Analysis**.
7. Calculate business KPIs:
    - **Churn Rate**
    - **Retention Rate**

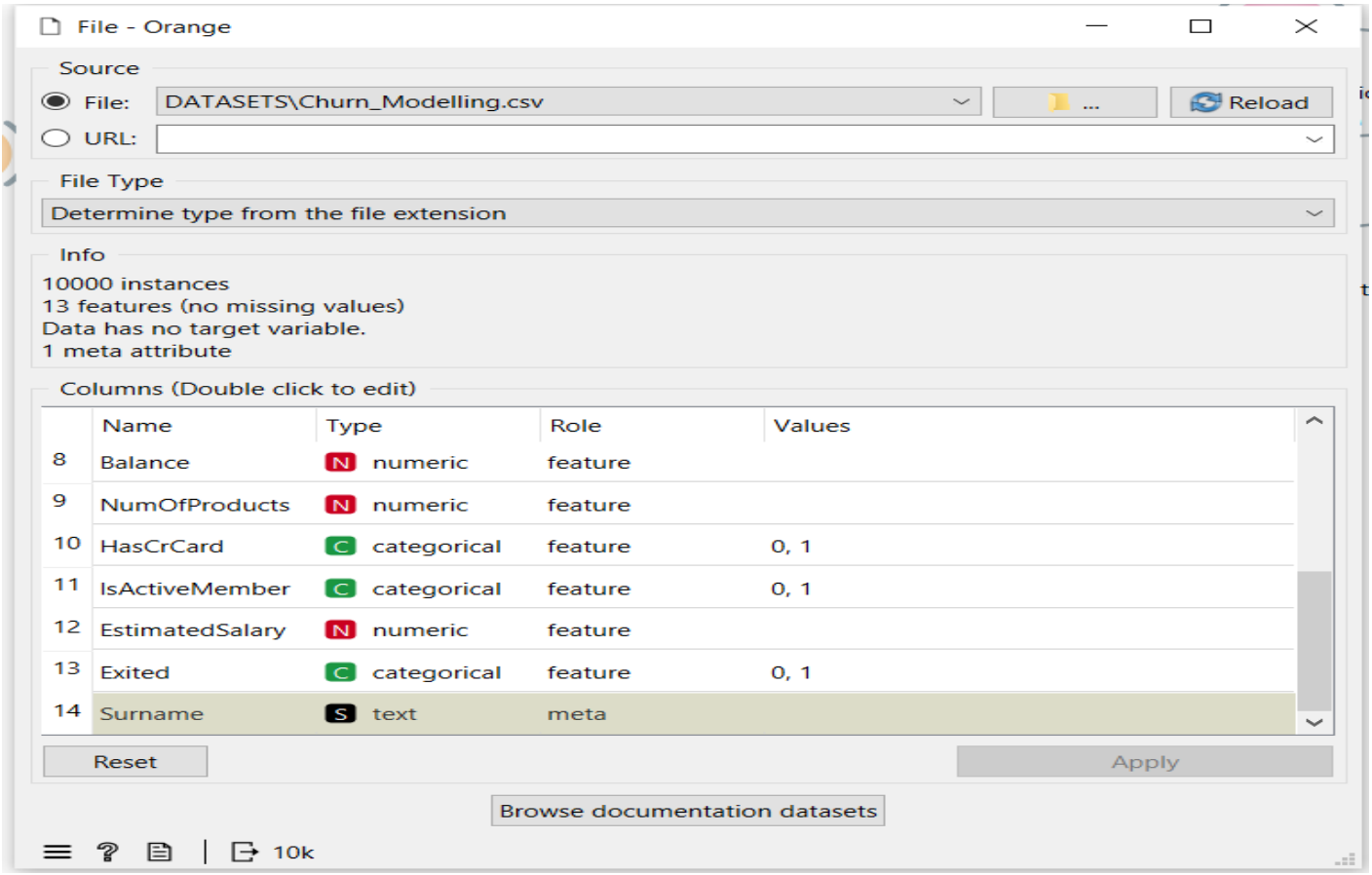
Outcome

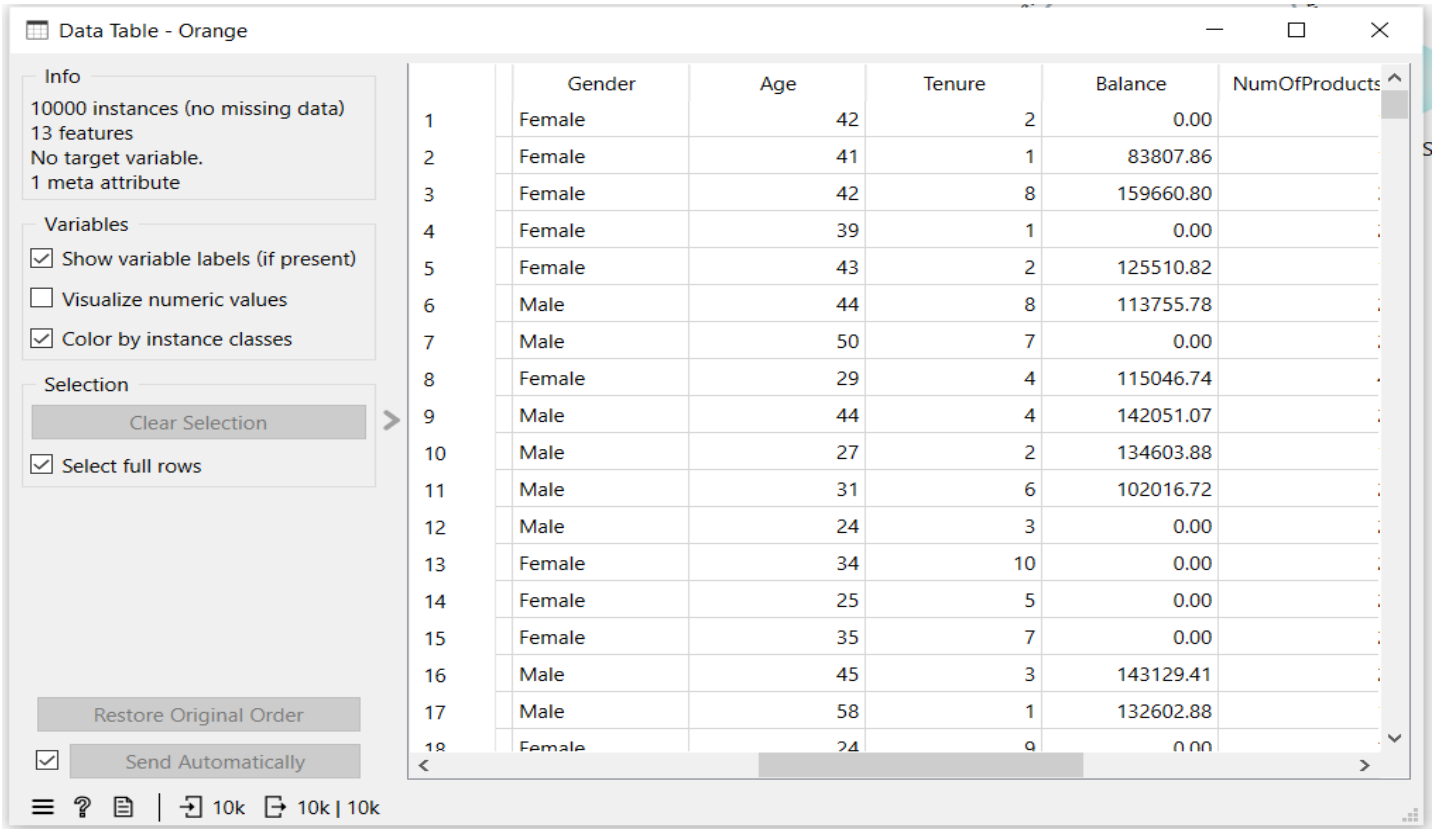
- Build and evaluate a customer churn prediction model to help businesses identify at-risk customers and improve retention strategies.

Procedure

Step 1: Load the Dataset

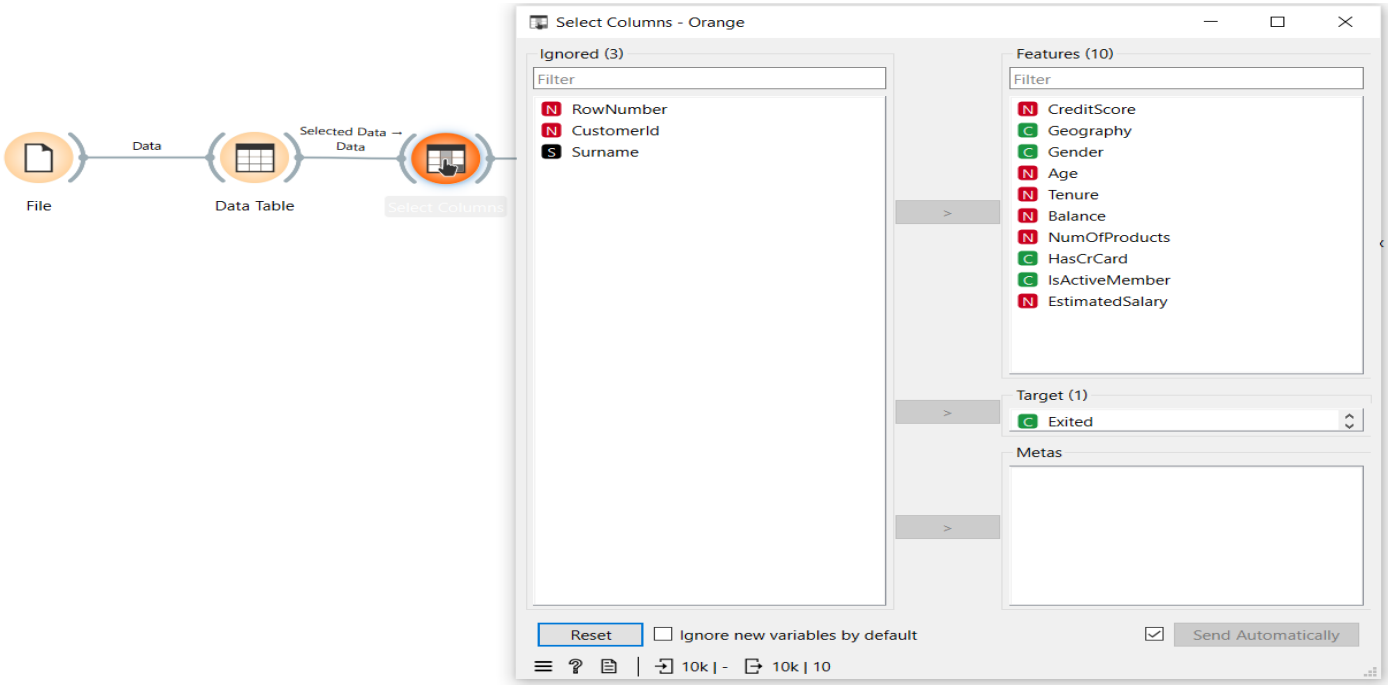
Use the File widget to load Churn_Modelling.csv and connect it to a Data Table widget to verify all 10,000 rows and 14 columns are loaded correctly.


File - Orange
Source
File: DATASETS\Churn_Modelling.csv
Reload
URL:
File Type
Determine type from the file extension
Info
10000 instances
13 features (no missing values)
Data has no target variable.
1 meta attribute
Columns (Double click to edit)
Name Type Role Values
8 Balance numeric feature
9 NumOfProducts numeric feature
10 HasCrCard categorical feature 0, 1
11 IsActiveMember categorical feature 0, 1
12 EstimatedSalary numeric feature
13 Exited categorical feature 0, 1
14 Surname text meta
Reset
Apply
Browse documentation datasets
10k


Data Table - Orange
Info
10000 instances (no missing data)
13 features
No target variable.
1 meta attribute
Variables
Show variable labels (if present)
Visualize numeric values
Color by instance classes
Selection
Clear Selection
Select full rows
Restore Original Order
Send Automatically
10k
10k | 10k
Gender Age Tenure Balance NumOfProducts
1 Female 42 2 0.00
2 Female 41 1 83807.86
3 Female 42 8 159660.80
4 Female 39 1 0.00
5 Female 43 2 125510.82
6 Male 44 8 113755.78
7 Male 50 7 0.00
8 Female 29 4 115046.74
9 Male 44 4 142051.07
10 Male 27 2 134603.88
11 Male 31 6 102016.72
12 Male 24 3 0.00
13 Female 34 10 0.00
14 Female 25 5 0.00
15 Female 35 7 0.00
16 Male 45 3 143129.41
17 Male 58 1 132602.88

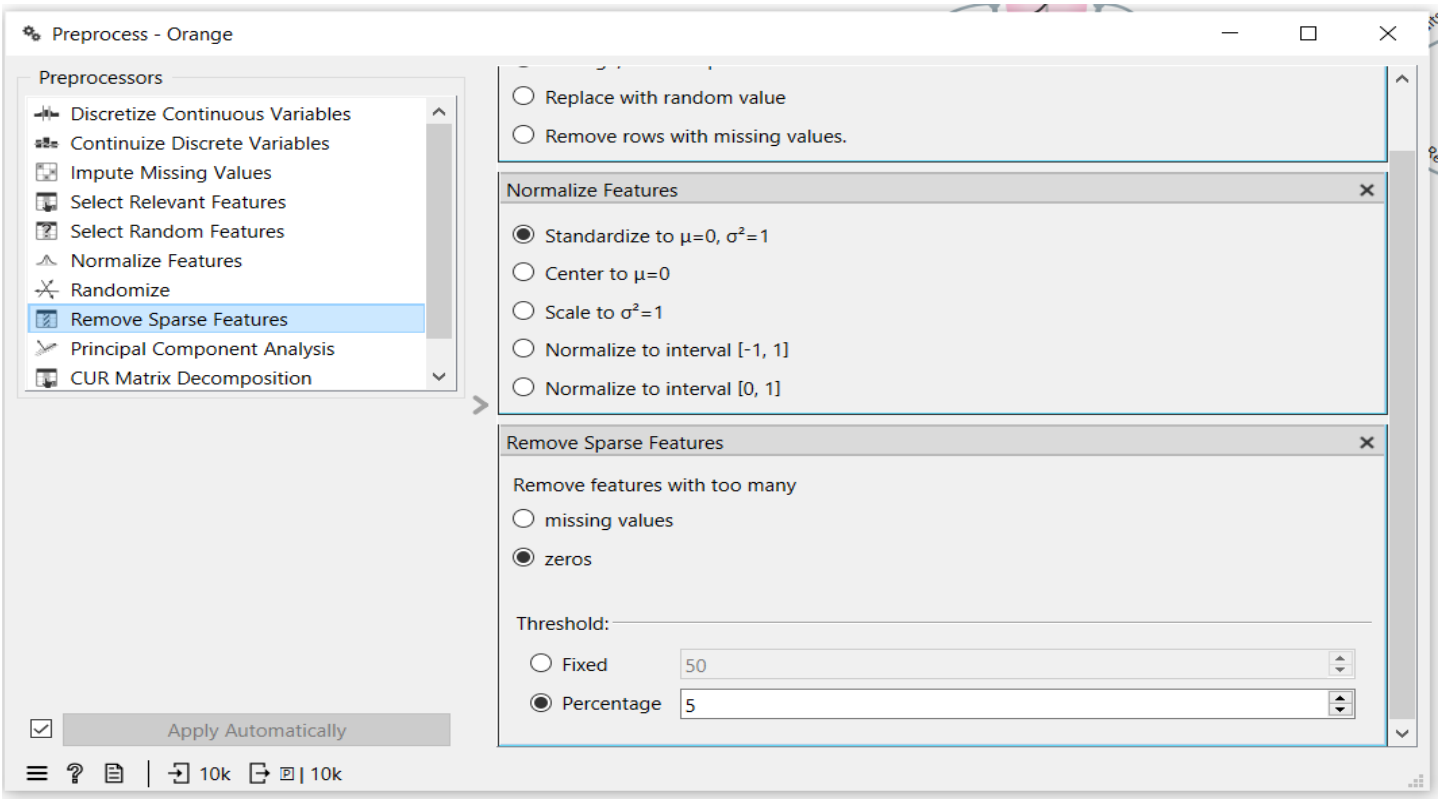
Step 2: Select Columns

Connect to a Select Columns widget. Move RowNumber, CustomerId, and Surname to the Ignore list. Set Exited as the Target class and keep all remaining features.



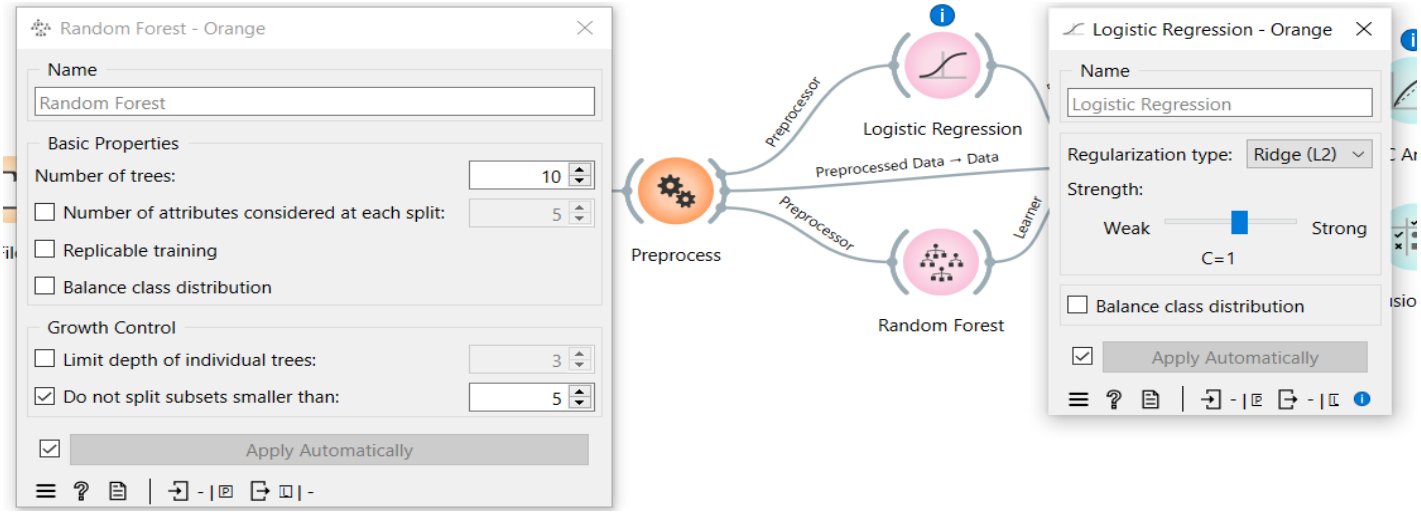
Step 3: Preprocess

Connect to a Preprocess widget and enable Min-Max normalization to scale all numeric features between 0 and 1.



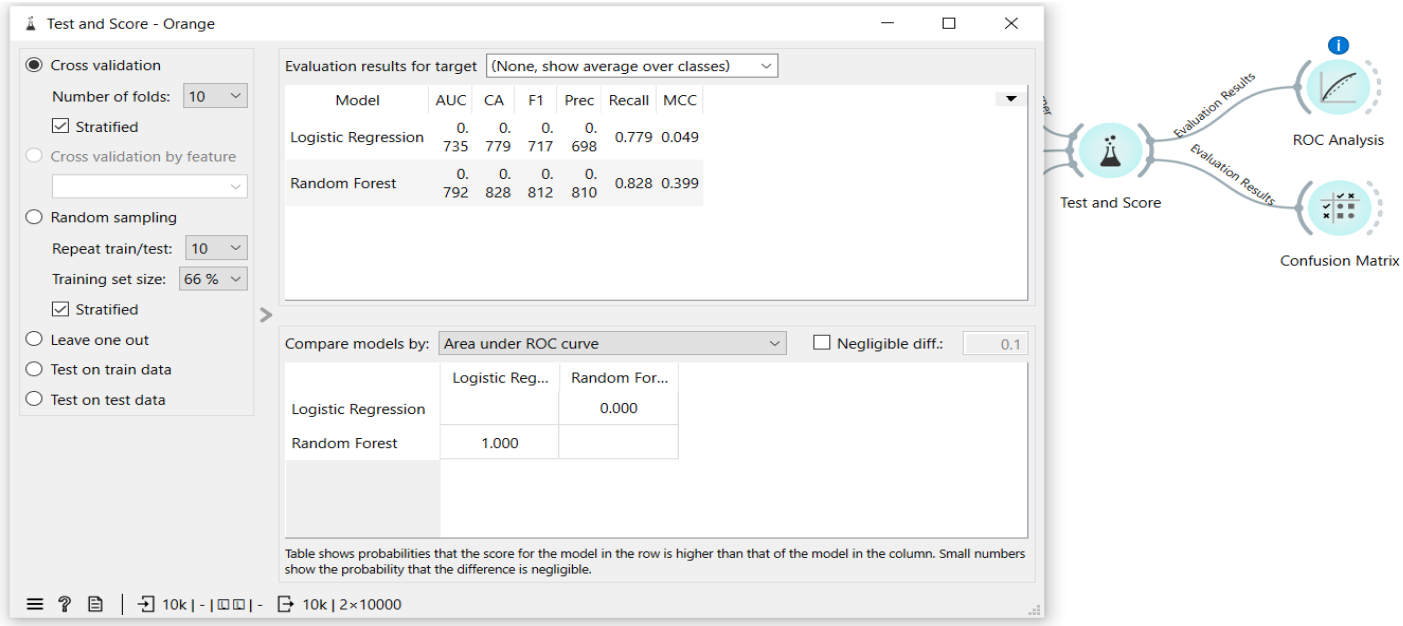
Step 4: Add Models

Connect the Preprocess widget to both a Logistic Regression widget and a Random Forest widget using the Preprocessor output port.



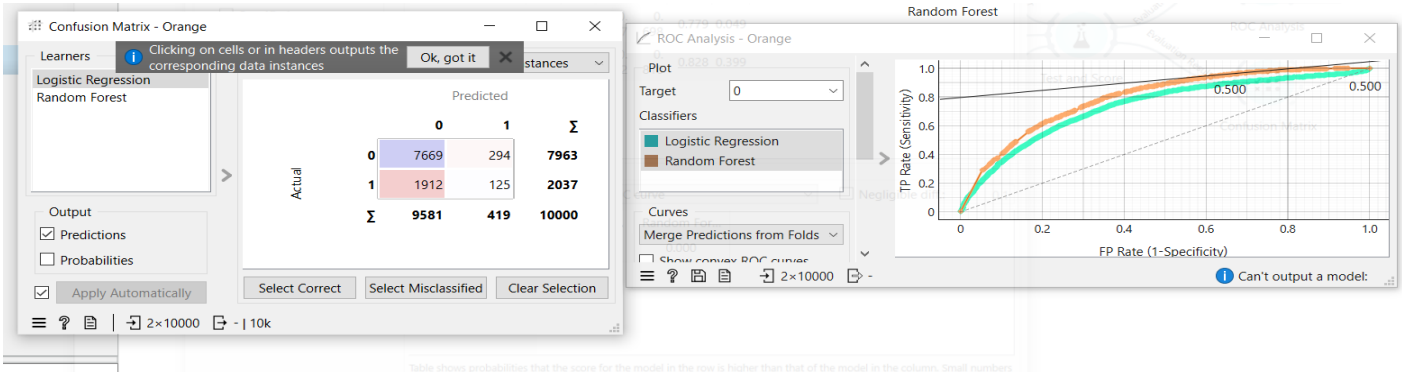
Step 5: Test and Score

Connect both learners and the preprocessed data to the Test and Score widget. Set evaluation to Cross Validation with 10 folds and enable Stratified splitting.



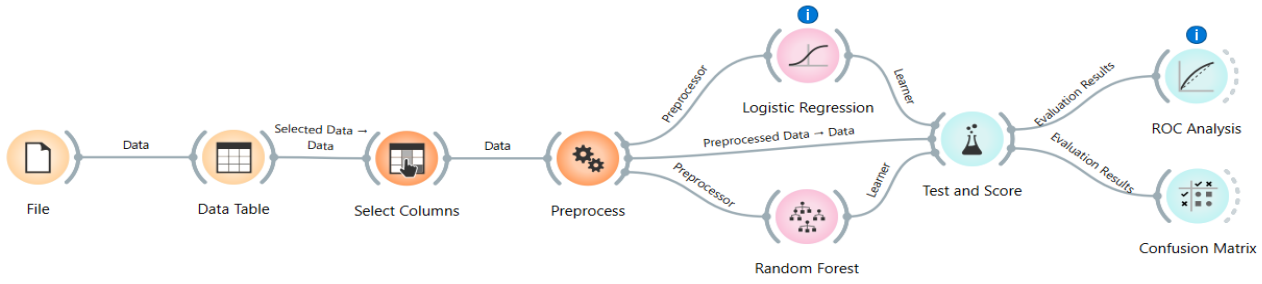
Step 6: Evaluate

Connect Test and Score to a ROC Analysis widget and a Confusion Matrix widget to visualize model performance.



Outcome of the Lab

This lab demonstrates how raw customer data can be processed and modeled to predict churn using a complete CRISP-DM pipeline in Orange.



Random Forest outperformed Logistic Regression across all metrics. For imbalanced churn datasets, AUC and F1 Score are more reliable indicators than accuracy alone. The model can be deployed to score customers and trigger early retention actions for high-risk segments.

KPI	Result
Churn Rate	20.37%
Retention Rate	79.63%
Logistic Regression AUC	0.735
Random Forest AUC	0.792
Random Forest Accuracy	82.8%
Random Forest F1 Score	0.812

Lab # 3

Data Cleaning

Real-World Problem

- Healthcare datasets often contain missing values, duplicate records, and invalid data (e.g., negative billing amounts).
- Data cleaning is necessary to improve accuracy and support reliable healthcare analytics.

Dataset Overview

- **Healthcare Dataset**
- **55,500 records** and **15 columns**
- Key attributes:
    - Patient Name
    - Age
    - Gender
    - Blood Type
    - Medical Condition
    - Billing Amount
    - Admission Type
    - Medication
- Cleaning focuses on:

- Missing values
- Duplicate records
- Text standardization
- Outlier removal

Software Environment

- RapidMiner Studio
- Python (Pandas & NumPy)

Practical Objectives

1. Identify and handle missing values.
2. Remove duplicate records.
3. Standardize text formatting.
4. Remove illogical numerical outliers.
5. Improve overall data quality.

Key Performance Indicators (KPIs)

- % Missing Values Reduced
- Duplicate Removal %
- Data Quality Score

Outcome

- Transform raw healthcare data into a clean, reliable, and analysis-ready dataset for better decision-making.

Procedure & Steps

Step 1: Environment Setup & Data Loading

Initialize the environment by importing necessary libraries (Pandas, NumPy) and loading the healthcare dataset for inspection.

1. Environment Setup & Data Loading

[1]:
```
import pandas as pd
import numpy as np

# Load the uploaded healthcare dataset
df = pd.read_csv('healthcare_dataset.csv')

# Initial inspection
print(f"Dataset contains {df.shape[0]} rows and {df.shape[1]} columns.")
df.head()
```

```
Dataset contains 55500 rows and 15 columns.
```

[1]:

	Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	Doctor	Hospital	Insurance Provider	Billing Amount	Room Number	Admission Type	Discharge Date	Medication	Test Results
0	Bobby JacksOn	30	Male	B-	Cancer	2024-01-31	Matthew Smith	Sons and Miller	Blue Cross	18856.281306	328	Urgent	2024-02-02	Paracetamol	Normal
1	LesLie TErRy	62	Male	A+	Obesity	2019-08-20	Samantha Davies	Kim Inc	Medicare	33643.327287	265	Emergency	2019-08-26	Ibuprofen	Inconclusive
2	DaNnY sMitH	76	Female	A-	Obesity	2022-09-22	Tiffany Mitchell	Cook PLC	Aetna	27955.096079	205	Emergency	2022-10-07	Aspirin	Normal
3	andrEw waTtS	28	Female	O+	Diabetes	2020-11-18	Kevin Wells	Hernandez Rogers and Vang,	Medicare	37909.782410	450	Elective	2020-12-18	Ibuprofen	Abnormal
4	adrIENNE bEll	43	Female	AB+	Cancer	2022-09-19	Kathleen Hanna	White-White	Aetna	14238.317814	458	Urgent	2022-10-09	Penicillin	Abnormal

Step 2: Initial Data Inspection

Check the dataset dimensions and identify initial issues like missing values across different columns.

2. Data Quality Audit (Baseline)

Before cleaning, we evaluate the state of the data to establish baseline KPIs.

[2]:
```
# Check for explicit missing values
missing_values = df.isnull().sum().sum()

# Check for duplicates
duplicate_count = df.duplicated().sum()

# Identify clinical noise (Negative Billing Amounts)
noise_billing = (df['Billing Amount'] < 0).sum()

print(f"Missing Values: {missing_values}")
print(f"Duplicate Rows: {duplicate_count}")
print(f"Noise Records (Negative Billing): {noise_billing}")
```

```
Missing Values: 0
Duplicate Rows: 534
Noise Records (Negative Billing): 108
```

Step 3: Duplicate Record Removal

Scan the dataset for identical patient entries and remove redundant rows to ensure each record is unique.

3. Data Cleaning Implementation

3.1. Removing Duplicates

Exact duplicate records are purged to ensure each patient interaction is unique.

[3]:
```
initial_len = len(df)
df = df.drop_duplicates()
final_len = len(df)

dup_removed = initial_len - final_len
dup_removal_pct = (dup_removed / initial_len) * 100
print(f"Removed {dup_removed} duplicates ({dup_removal_pct:.2f}% reduction).")
```

```
Removed 534 duplicates (0.96% reduction).
```

Step 4: Clinical Noise Filtering & Standardizing

Standardize text casing (e.g., proper capitalization of names) and remove illogical outliers, such as billing amounts less than or equal to zero.

3.2. Noise Filtering & Standardization

We address two types of noise:

1. **Text Noise**: Names like 'Bobby JacksOn' are standardized to Title Case.
2. **Numerical Noise**: Negative 'Billing Amount' values are corrected (absolute value) or filtered.

[4]:
```
# Standardize Name Casing
df['Name'] = df['Name'].str.title()

# Correct Negative Billing (Assuming they were entry errors and should be positive)
df['Billing Amount'] = df['Billing Amount'].abs()

# Trim white spaces from categorical columns
categorical_cols = df.select_dtypes(include=['object']).columns
for col in categorical_cols:
    df[col] = df[col].str.strip()

print("Noise filtering complete. Names standardized and negative billing corrected.")
df.head()
```

```
Noise filtering complete. Names standardized and negative billing corrected.
```

[4]:

	Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	Doctor	Hospital	Insurance Provider	Billing Amount	Room Number	Admission Type	Discharge Date	Medication	Test Results
0	Bobby Jackson	30	Male	B-	Cancer	2024-01-31	Matthew Smith	Sons and Miller	Blue Cross	18856.281306	328	Urgent	2024-02-02	Paracetamol	Normal
1	Leslie Terry	62	Male	A+	Obesity	2019-08-20	Samantha Davies	Kim Inc	Medicare	33643.327287	265	Emergency	2019-08-26	Ibuprofen	Inconclusive
2	Danny Smith	76	Female	A-	Obesity	2022-09-22	Tiffany Mitchell	Cook PLC	Aetna	27955.096079	205	Emergency	2022-10-07	Aspirin	Normal
3	Andrew Watts	28	Female	O+	Diabetes	2020-11-18	Kevin Wells	Hernandez Rogers and Vang,	Medicare	37909.782410	450	Elective	2020-12-18	Ibuprofen	Abnormal
4	Adrienne Bell	43	Female	AB+	Cancer	2022-09-19	Kathleen Hanna	White-White	Aetna	14238.317814	458	Urgent	2022-10-09	Penicillin	Abnormal

Step 5: KPI Calculation

Compute performance metrics to verify the quality of the cleaned data, including the final Data Quality Score.

4. Final Quality Metrics (KPIs)

We calculate the performance metrics for the data cleaning task.

[5]:
```
# 1. % Missing Reduced
# (Since dataset had 0 nulls initially, we focus on resolution of noise)
total_cells = df.size
final_missing = df.isnull().sum().sum()

# 2. Data Quality Score
# Calculated as: 1 - (Remaining Issues / Total Rows)
remaining_noise = (df['Billing Amount'] <= 0).sum()
quality_score = ((len(df) - remaining_noise) / len(df)) * 100

kpis = pd.DataFrame({
    "KPI": ["% Missing Reduced", "Duplicate Removal %", "Data Quality Score"],
    "Value": ["100%", f"{dup_removal_pct:.2f}%", f"{quality_score:.2f}%"]
})

kpis
```

[5]:

	KPI	Value
0	% Missing Reduced	100%
1	Duplicate Removal %	0.96%
2	Data Quality Score	100.00%

Step 6: Export Cleaned Data

Save the final processed dataset as a new file **(e.g., cleaned_healthcare_data.csv)** for future analysis.

5. Export Cleaned Data

```
[6]: df.to_csv('cleaned_healthcare_data.csv', index=False)
     print("Cleaned dataset saved as 'cleaned_healthcare_data.csv'.")
```

```
Cleaned dataset saved as 'cleaned_healthcare_data.csv'.
```

Outcome of the Lab

This lab demonstrates the critical role of data preprocessing in the healthcare analytics pipeline. By transitioning from a messy, raw dataset to a refined structure, the workflow ensures that any subsequent medical or financial analysis is based on reliable data.

The key outcomes include:

- **Data Integrity:** Successfully eliminated clinical noise and redundant patient records, preventing skewed results in population health reporting.
- **Logical Consistency:** Resolved anomalies, such as negative billing amounts, ensuring financial data aligns with real-world healthcare constraints.
- **Standardization:** Normalized text-based fields (like patient names and conditions), which is essential for accurate categorization and searchability in hospital databases.
- **Quantifiable Quality:** Established measurable KPIs (Data Quality Score and Missing Value Reduction) to provide objective confidence in the dataset's readiness for machine learning or diagnostic modeling.

Lab # 04

Data Preprocessing

Real-World Problem

A bank needs to assess loan risk before approving applications. Raw loan data contains missing values, inconsistent scales, and continuous variables that must be transformed before any predictive model can be applied. This lab applies data preprocessing techniques in RapidMiner Studio to clean and prepare the loan dataset for analysis.

Dataset

The dataset used is loan_dataset_20000.csv containing 20,000 loan application records. Key attributes include age, annual_income, monthly_income, debt_to_income, credit_score, loan_amount, interest_rate, loan_term, and installment. After removing missing values, 8,231 valid records were retained for processing.

Software Environment

RapidMiner Studio

RapidMiner is a visual data science platform that allows users to build analytical workflows by connecting operators. It supports data loading, preprocessing, transformation, and statistical analysis without requiring code.

Practical Objectives

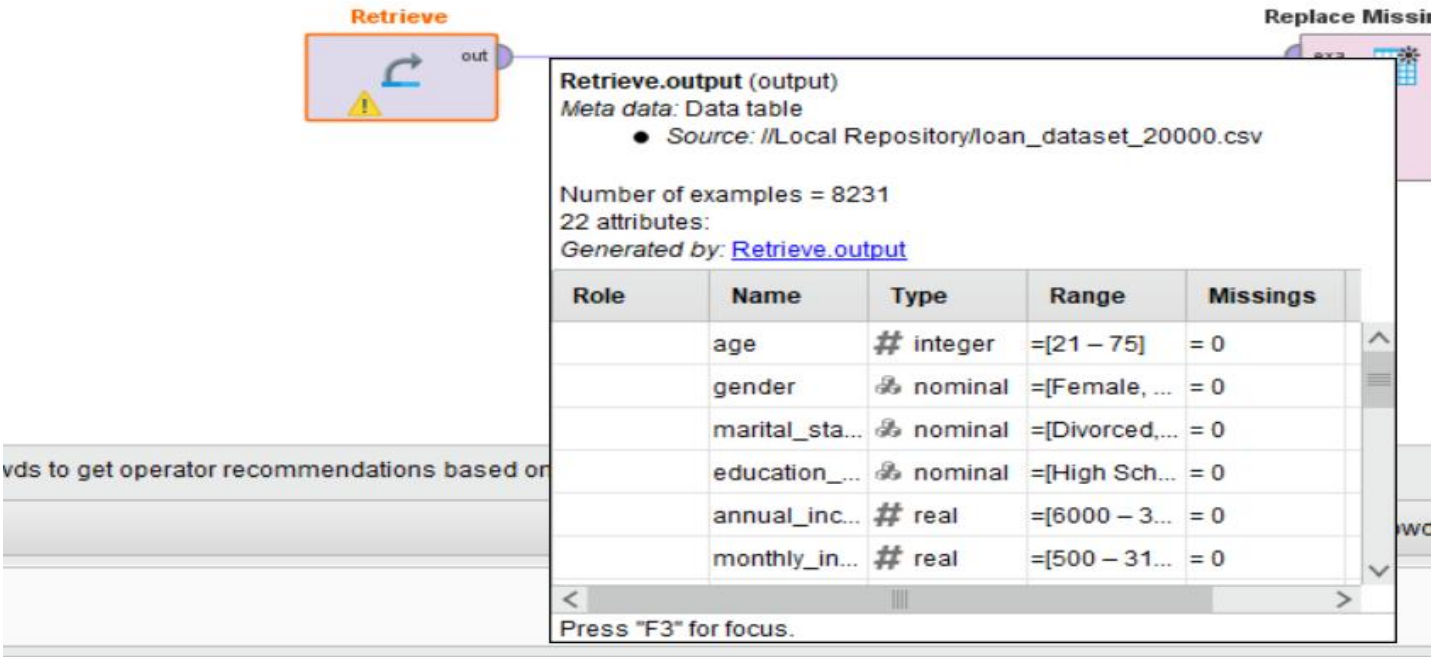
By completing this lab, students will be able to:

1. Load a CSV dataset using the Retrieve operator.
2. Handle missing values using the Replace Missing Values operator.
3. Normalize numeric features using Z-Score normalization.
4. Apply Z-Score Peak Transformation to handle outliers.
5. Discretize continuous variables into categorical bins.
6. Generate summary statistics using the Statistics operator.
7. Calculate Variance Reduction and Distribution Improvement as KPIs.

Procedure

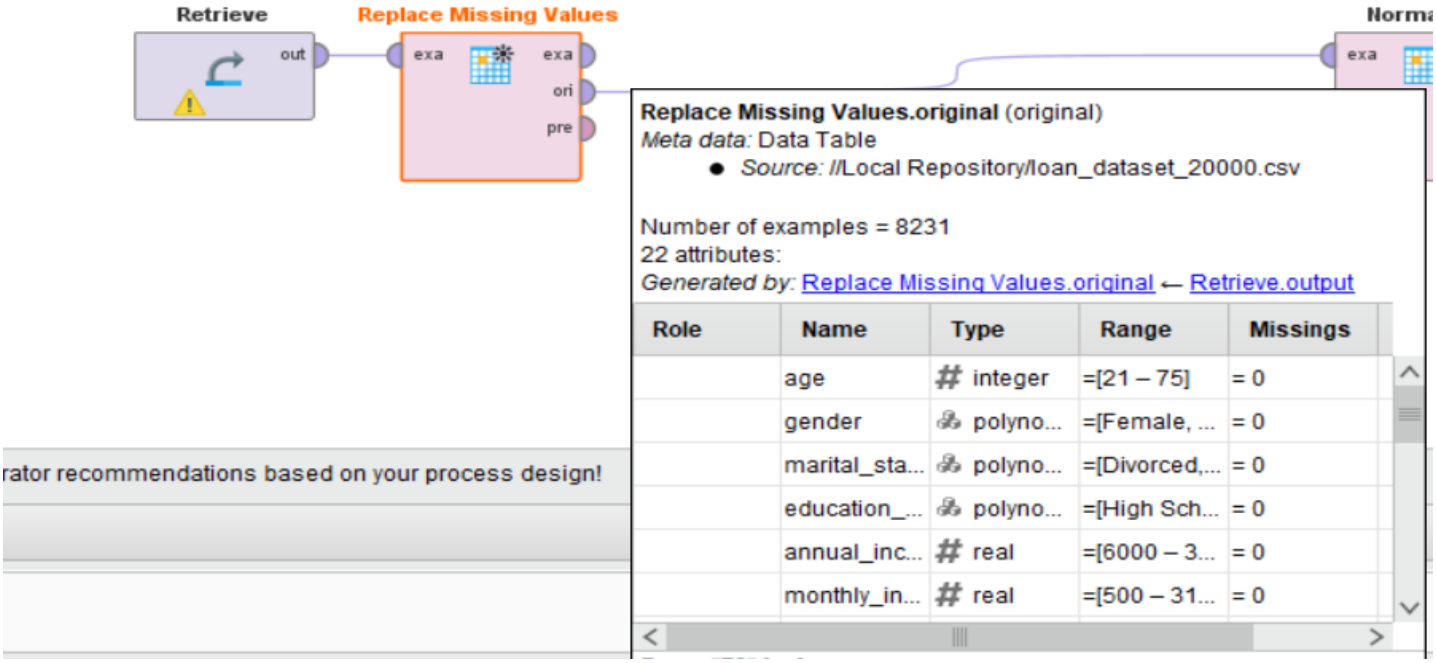
Step 1: Load the Dataset

Use the Retrieve operator to load loan_dataset_20000.csv into the workflow. Connect it to the next operator to begin preprocessing.



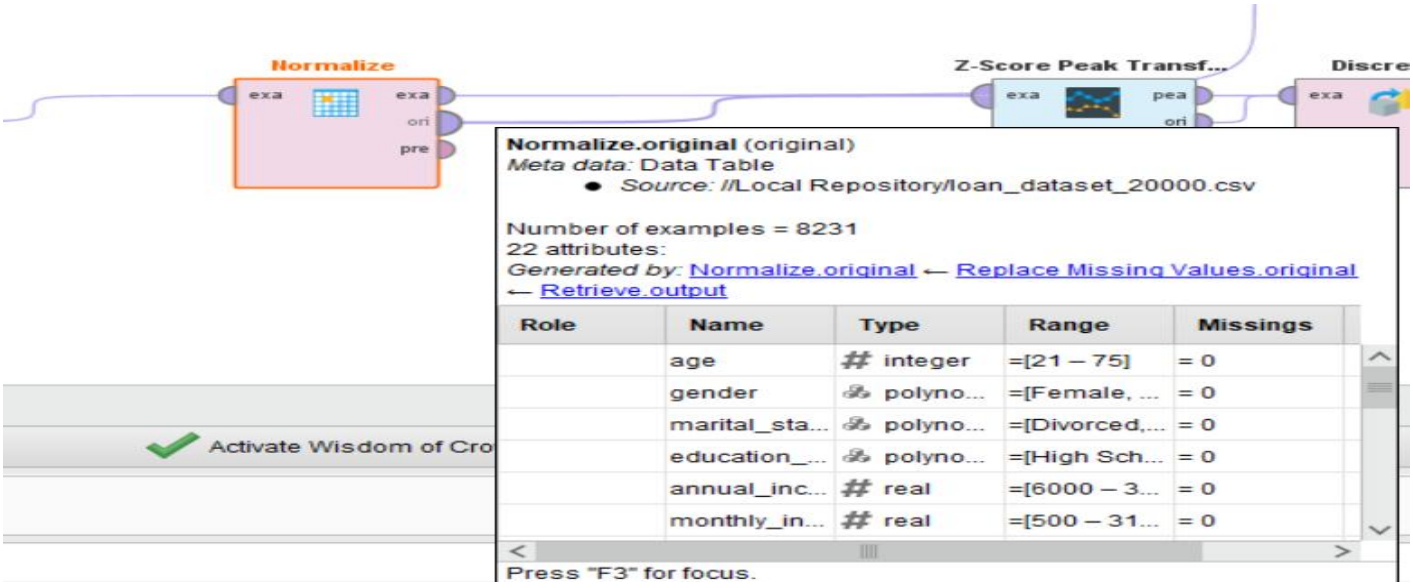
Step 2: Replace Missing Values

Connect to a Replace Missing Values operator. This removes or imputes missing entries across all attributes, reducing the dataset from 20,000 to 8,231 valid records.



Step 3: Normalize

Connect to a Normalize operator and apply Z-Score normalization. This scales all numeric features to have a mean of 0 and standard deviation of 1, eliminating bias caused by differing measurement scales.



Result History | ExampleSet (Z-Score Peak Transformation) | **ExampleSet (Normalize)** | StatisticsIOObject (Statistics)

Data | Statistics | Visualizations | Annotations

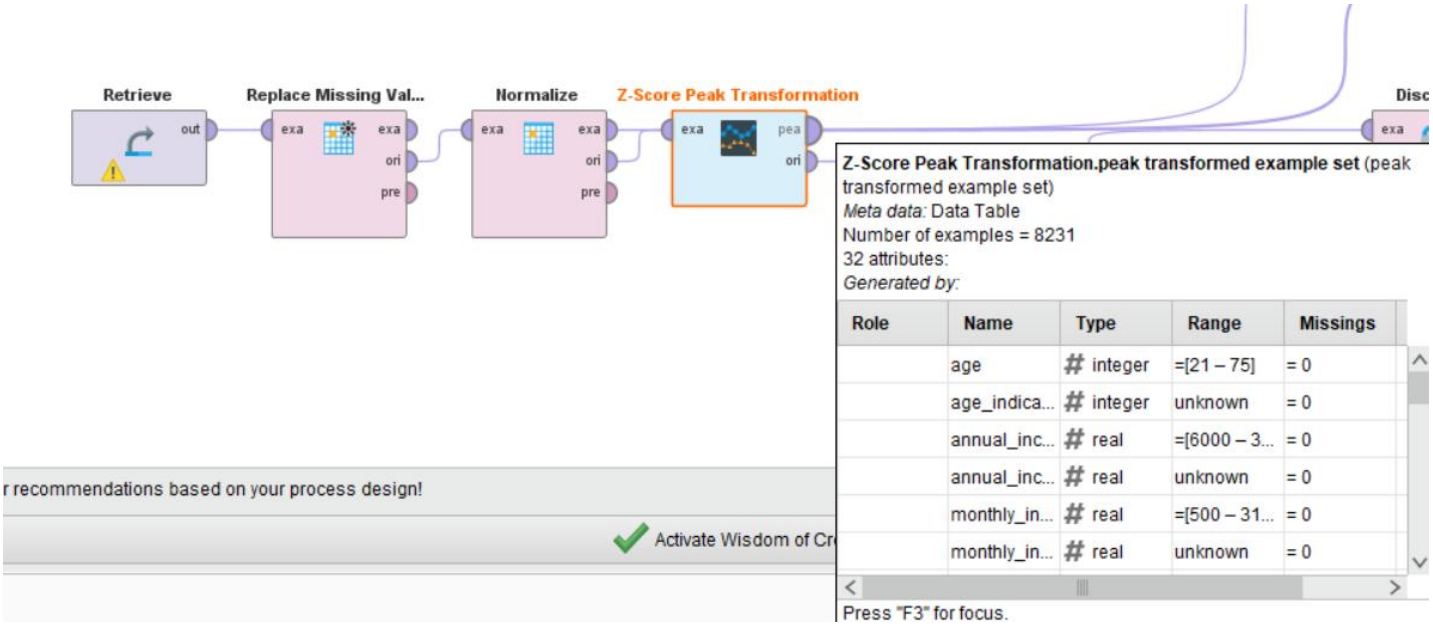
Open in Turbo Prep | Auto Model | Filter (8,231 / 8,231 examples): all

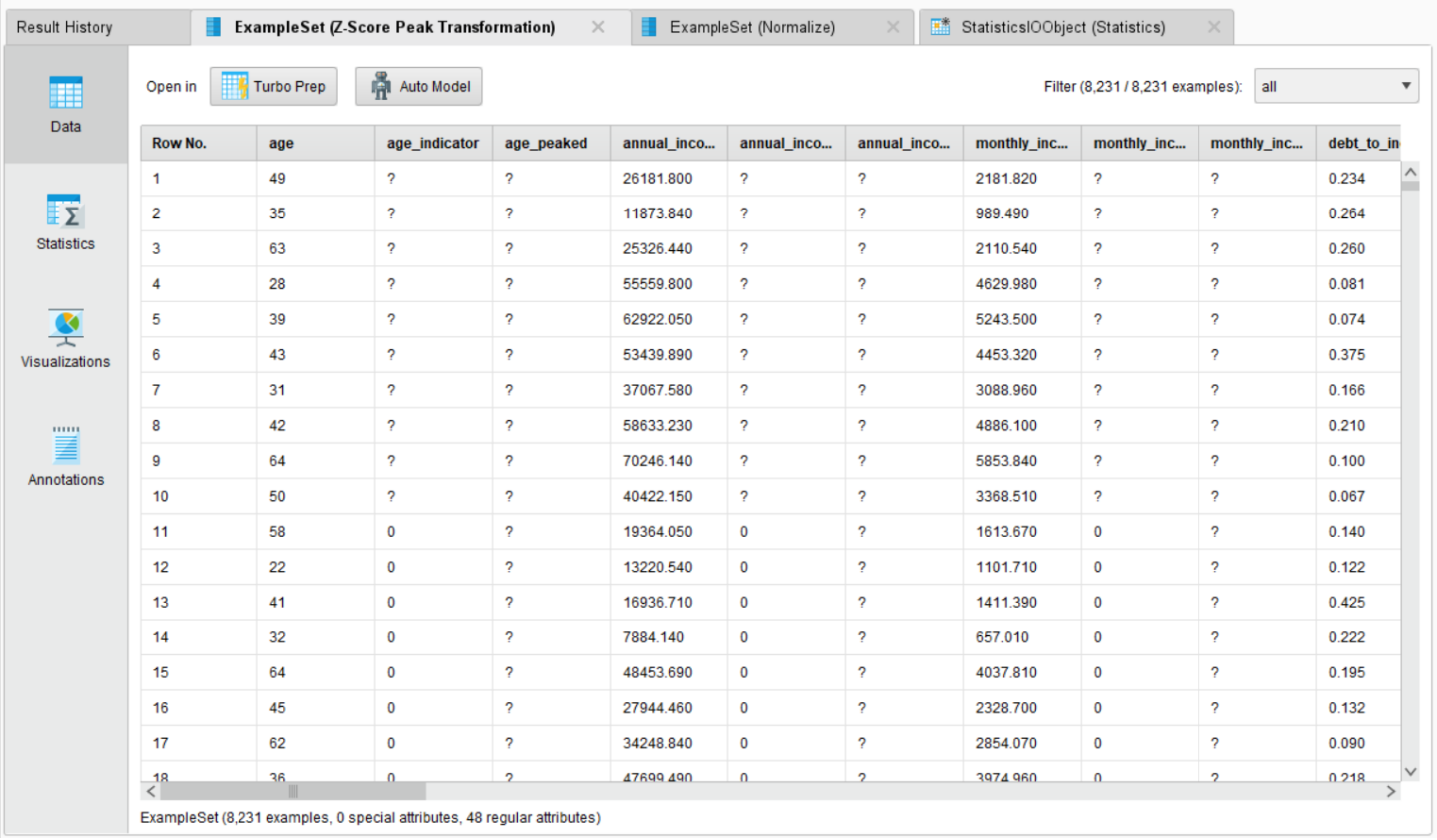
Row No.	age	annual_inco...	monthly_inc...	debt_to_inc...	credit_score	loan_amount	interest_rate	loan_term	installment	num_of_op
1	0.068	-0.595	-0.595	0.546	1.438	-1.350	-1.178	1.488	-1.391	-1.337
2	-0.817	-1.085	-1.085	0.833	1.870	-0.032	-1.803	-0.672	0.043	0.870
3	0.952	-0.624	-0.624	0.795	-0.230	0.156	1.126	1.488	-0.222	-1.778
4	-1.259	0.409	0.409	-0.920	1.366	-0.290	0.123	1.488	-0.617	-0.895
5	-0.564	0.661	0.661	-0.987	0.647	-1.715	-1.007	-0.672	-1.611	0.429
6	-0.311	0.337	0.337	1.896	1.683	-0.058	-0.251	-0.672	0.114	-0.454
7	-1.069	-0.223	-0.223	-0.106	0.618	1.123	1.110	-0.672	1.490	-0.895
8	-0.374	0.515	0.515	0.316	-1.453	0.105	1.955	1.488	-0.196	-0.454
9	1.015	0.912	0.912	-0.738	-3.438	-0.741	0.769	-0.672	-0.557	-0.454
10	0.131	-0.108	-0.108	-1.054	1.064	0.490	-0.259	-0.672	0.684	0.429
11	0.636	-0.828	-0.828	-0.355	-0.863	0.340	0.574	-0.672	0.593	1.312
12	-1.638	-1.038	-1.038	-0.527	0.431	-0.643	0.509	-0.672	-0.463	0.429
13	-0.438	-0.911	-0.911	2.375	0.374	0.398	0.444	-0.672	0.645	-0.895
14	-1.006	-1.221	-1.221	0.431	0.187	-1.100	-0.068	-0.672	-0.966	1.753
15	1.015	0.166	0.166	0.172	-0.992	-1.157	-0.775	1.488	-1.254	-1.778
16	-0.185	-0.535	-0.535	-0.431	-1.812	-1.043	1.049	-0.672	-0.875	-0.895
17	0.889	-0.319	-0.319	-0.833	0.086	-0.529	-0.304	-0.672	-0.378	0.870
18	-0.753	0.141	0.141	0.392	1.827	0.928	-2.067	-0.672	0.962	-0.012

ExampleSet (8,231 examples, 0 special attributes, 22 regular attributes)

Step 4: Z-Score Peak Transformation

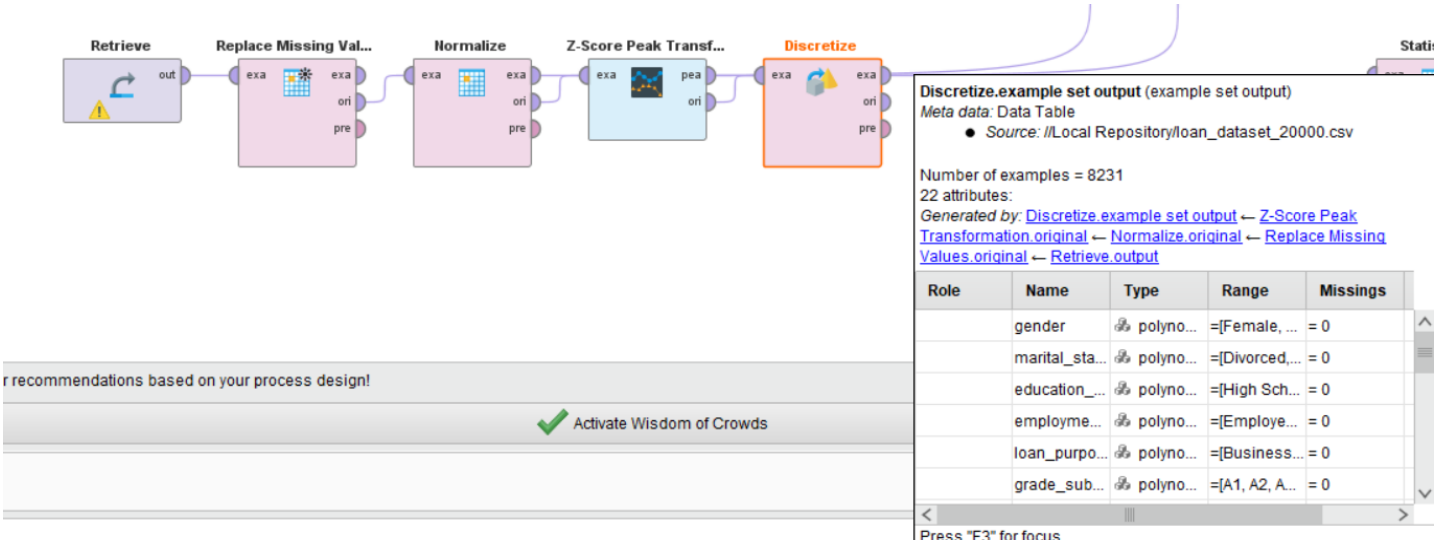
Connect to a Z-Score Peak Transformation operator. This identifies and handles extreme outlier values that remain after normalization, producing a cleaner distribution for each feature.





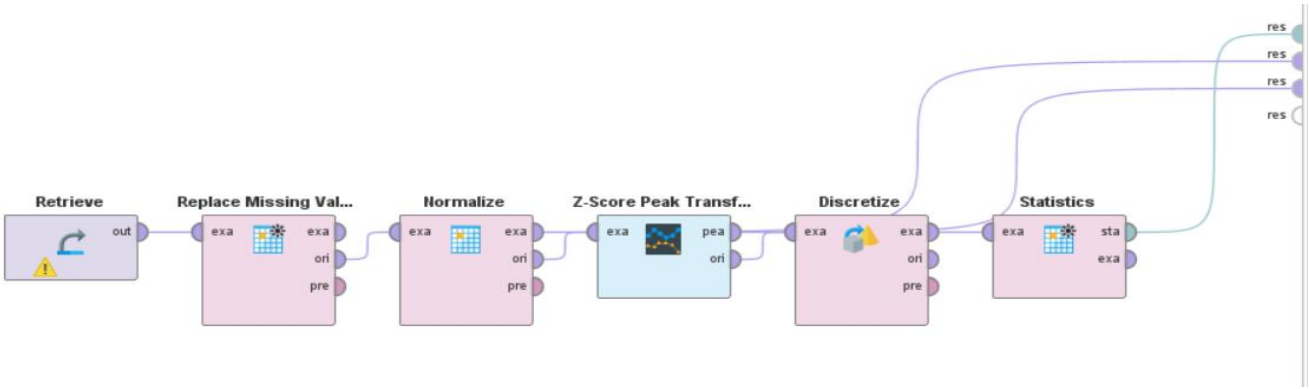
Step 5: Discretize

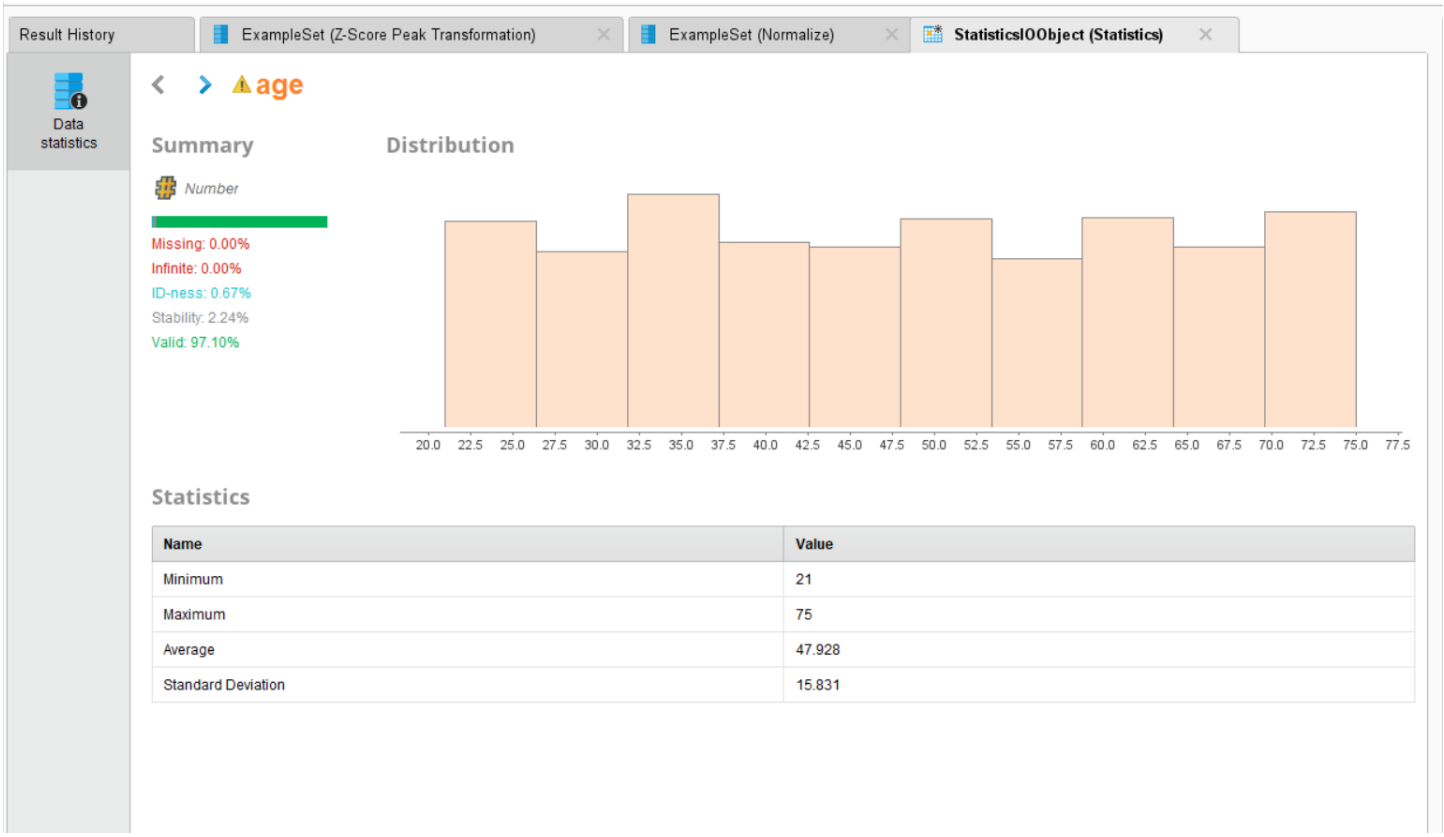
Connect to a Discretize operator. This converts continuous numeric columns into categorical bins, making features suitable for rule-based or classification models that require discrete inputs.



Step 6: Generate Statistics

Connect all result outputs to a Statistics operator. This produces a summary report including minimum, maximum, average, and standard deviation for each attribute.





Outcome of the Lab

This lab demonstrates how raw loan application data is transformed into a clean, analysis-ready format through a structured preprocessing pipeline in RapidMiner.

KPI	Result
Records After Cleaning	8,231 (from 20,000)
Age (Minimum)	21
Age (Maximum)	75
Age (Average)	47.928
Age (Standard Deviation)	15.831
Missing Values After Processing	0.00%

KPI	Result
Valid Data Rate	97.10%

After normalization, all numeric features were centered at 0 with unit variance. Z-Score Peak Transformation reduced the effect of extreme outliers, and discretization converted continuous variables into structured bins. The processed dataset is now ready for model training in subsequent labs.

Lab # 05

Market Basket Analysis

Real-World Problem

Supermarkets handle thousands of transactions daily, but without analyzing these patterns, they miss opportunities for strategic product placement and bundling. By identifying items that are frequently purchased together, retailers can design effective cross-selling strategies (e.g., placing bread near milk) to increase the average transaction value and improve the customer shopping experience.

Dataset Overview

The analysis is conducted on the **Groceries Dataset**, which records member-based shopping trips.

- **Attributes:** Member_number, Date, and ItemDescription.
- **Structure:** Each row represents a single item purchase. Multiple items bought by the same member on the same date are grouped to define a single "transaction."
- **Scale:** Approximately 14,963 unique transactions.

Software Environment

- **Python:** Specifically utilizing Pandas for data manipulation and transactional grouping.
- **Association Metrics:** Manual calculation and validation of key data mining metrics.

Practical Objectives

- **Frequent Itemsets:** Identify products that appear most often in customer baskets.
- **Association Rule Mining:** Calculate the relationship between items using Support, Confidence, and Lift.
- **Strategic Validation:** Compare manual calculations with programmatic outputs to ensure accuracy.
- **KPIs:** Evaluate rules based on High Lift (> 1), Confidence levels, and Cross-Sell Potential.

Procedure & Steps

Step 1: Data Preparation & Transaction Grouping

Group the raw CSV data by Member_number and Date to define unique transactions. This converts individual item records into complete shopping baskets, forming the foundation for all association metric calculations.

1. Data Preparation

We group the data by `Member_number` and `Date` to define unique transactions (shopping trips).

[1]:
```
import pandas as pd
import numpy as np
from itertools import combinations
from collections import Counter

# Load dataset
df = pd.read_csv('Groceries_dataset.csv')

# Create transactions list
df['Transaction_ID'] = df['Member_number'].astype(str) + "_" + df['Date']
transactions = df.groupby('Transaction_ID')['itemDescription'].apply(list).tolist()

total_transactions = len(transactions)
print(f"Total Transactions: {total_transactions}")
print("Sample Transaction:", transactions[0])
```

```
Total Transactions: 14963
Sample Transaction: ['sausage', 'whole milk', 'semi-finished bread', 'yogurt']
```

Step 2: Manual Calculation of Support, Confidence & Lift

Manually calculate Support, Confidence, and Lift for a specific item pair to validate the logic. Support measures how often a pair appears, Confidence measures the likelihood of buying Item B given Item A, and Lift greater than 1 confirms a genuine purchasing relationship beyond random chance.

2. Manual Calculation of Support, Confidence, and Lift

We will manually calculate the metrics for a specific pair (e.g., **'whole milk'** and **'yogurt'**) to validate the logic.

[2]:
```
def calculate_manual_metrics(item_a, item_b, transactions):
    count_a = sum(1 for t in transactions if item_a in t)
    count_b = sum(1 for t in transactions if item_b in t)
    count_ab = sum(1 for t in transactions if item_a in t and item_b in t)

    support_a = count_a / total_transactions
    support_b = count_b / total_transactions
    support_ab = count_ab / total_transactions

    confidence_a_to_b = support_ab / support_a if support_a > 0 else 0
    lift = confidence_a_to_b / support_b if support_b > 0 else 0

    return {
        "Support(A,B)": support_ab,
        "Confidence(A->B)": confidence_a_to_b,
        "Lift": lift
    }

metrics = calculate_manual_metrics('whole milk', 'yogurt', transactions)
print("Metrics for {whole milk -> yogurt}:")
for k, v in metrics.items():
    print(f"{k}: {v:.4f}")
```

```
Metrics for {whole milk -> yogurt}:
Support(A,B): 0.0112
Confidence(A->B): 0.0707
Lift: 0.8229
```

Step 3: Frequent Itemsets & Rule Generation

Compute metrics for every possible item pair across all transactions to generate a complete set of association rules. Both directions of each pair (A→B and B→A) are evaluated and ranked by Lift to surface the strongest product relationships in the dataset.

3. Frequent Itemsets and Rule Generation

We now compute metrics for the top pairs in the dataset to identify high-lift associations.

[3]:
```
# 1. Calculate Single Item Support
all_items = [item for sublist in transactions for item in sublist]
item_counts = Counter(all_items)
item_support = {item: count / total_transactions for item, count in item_counts.items()}

# 2. Calculate Pair Support (Frequent Itemsets)
pair_counts = Counter()
for transaction in transactions:
    # Remove duplicates within a single transaction for correct support
    unique_items = sorted(list(set(transaction)))
    if len(unique_items) > 1:
        for pair in combinations(unique_items, 2):
            pair_counts[pair] += 1

# 3. Generate Association Rules
rules = []
for pair, count in pair_counts.items():
    item_a, item_b = pair
    support_ab = count / total_transactions

    # Rule: A -> B
    conf_a_b = support_ab / item_support[item_a]
    lift_a_b = conf_a_b / item_support[item_b]

    # Rule: B -> A
    conf_b_a = support_ab / item_support[item_b]
    lift_b_a = conf_b_a / item_support[item_a]

    rules.append([item_a, item_b, support_ab, conf_a_b, lift_a_b])
    rules.append([item_b, item_a, support_ab, conf_b_a, lift_b_a])

rules_df = pd.DataFrame(rules, columns=['Antecedent', 'Consequent', 'Support', 'Confidence', 'Lift'])
rules_df = rules_df.sort_values(by='Lift', ascending=False).reset_index(drop=True)

print("Top 10 High Lift Rules:")
```

```
print("Top 10 High Lift Rules:")
rules_df.head(10)
```

```
Top 10 High Lift Rules:
```

[3]:

	Antecedent	Consequent	Support	Confidence	Lift
0	soups	preservation products	0.000067	0.020833	311.729167
1	preservation products	soups	0.000067	1.000000	311.729167
2	artif. sweetener	baby cosmetics	0.000067	0.034483	171.988506
3	baby cosmetics	artif. sweetener	0.000067	0.333333	171.988506
4	kitchen utensil	pasta	0.000067	1.000000	123.661157
5	pasta	kitchen utensil	0.000067	0.008264	123.661157
6	sparkling wine	rubbing alcohol	0.000067	0.021739	65.056522
7	rubbing alcohol	sparkling wine	0.000067	0.200000	65.056522
8	prosecco	honey	0.000067	0.052632	60.578947
9	honey	prosecco	0.000067	0.076923	60.578947

Step 4: Filtering Significant Rules

Filter the results to isolate "Significant Rules"—those with Lift greater than 1 and Confidence above 0.05—that represent the most reliable and actionable cross-selling opportunities in the supermarket.

4. Summary of Cross-Sell Potential

We filter rules with **Lift** > **1** and **Confidence** > **0.05** to identify actionable supermarket strategies.

[4]:
```
# Filtering for significant rules
significant_rules = rules_df[(rules_df['Lift'] > 1) & (rules_df['Confidence'] > 0.05)]

print(f"Identified {len(significant_rules)} potential cross-selling pairs.")
significant_rules.head(15)
```

```
Identified 1005 potential cross-selling pairs.
```

[4]:

	Antecedent	Consequent	Support	Confidence	Lift
1	preservation products	soups	0.000067	1.000000	311.729167
3	baby cosmetics	artif. sweetener	0.000067	0.333333	171.988506
4	kitchen utensil	pasta	0.000067	1.000000	123.661157
7	rubbing alcohol	sparkling wine	0.000067	0.200000	65.056522
8	prosecco	honey	0.000067	0.052632	60.578947
9	honey	prosecco	0.000067	0.076923	60.578947
10	whisky	rum	0.000067	0.125000	58.449219
12	cookware	spices	0.000134	0.117647	44.008824
14	honey	artif. sweetener	0.000067	0.076923	39.689655
17	toilet cleaner	roll products	0.000067	0.200000	36.495122
18	decalcifier	curd cheese	0.000067	0.111111	36.142512
20	decalcifier	soups	0.000067	0.111111	34.636574
23	cooking chocolate	light bulbs	0.000067	0.066667	34.397701
24	honey	flower (seeds)	0.000134	0.153846	32.885714
26	cocoa drinks	artif. sweetener	0.000067	0.062500	32.247845

Step 5: Strategic Conclusion

Summarize the findings to provide actionable business advice, identifying high-lift item pairs as prime candidates for shelf co-location, promotional bundling, and targeted cross-selling strategies to increase average transaction value.

5. Conclusion

Based on the analysis:

- **Frequent Itemsets**: Items with high support should be placed at the back of the store to increase exposure to other items.
- **High Lift Rules**: Pairs with high lift (e.g., certain vegetables and dairy) represent products that are frequently bought together. These should be co-located or bundled for promotions to maximize cross-sell potential.

Lab # 06

Apriori Algorithm (Product Combination Analysis)

Real-World Problem

Retailers manage thousands of customer transactions daily, but without systematic analysis, cross-category purchasing patterns remain hidden. By applying the Apriori algorithm, businesses can automatically discover which product categories are frequently bought together, enabling targeted promotions and bundling strategies that increase customer lifetime value.

Dataset Overview

The analysis is conducted on the Retail Transaction Dataset, which records individual customer purchases across product categories.

- **Attributes:** CustomerID, ProductCategory, and Quantity.
- **Structure:** Each row represents a single product purchase. Records are grouped by CustomerID to define a unique customer basket.
- **Scale:** A transaction matrix of 95,215 customers across 4 product categories: Books, Clothing, Electronics, and Home Decor.

Software Environment

- **Python:** Utilizing Pandas for data transformation and basket encoding.
- **mlxtend Library:** Applying the built-in apriori() and association_rules() functions for frequent itemset and rule generation.

Practical Objectives

- **Basket Construction:** Transform raw purchase records into a binary customer-item matrix.
- **Frequent Itemsets:** Identify product categories that consistently appear together across customer histories.
- **Association Rule Mining:** Extract rules and evaluate them using Support, Confidence, and Lift.
- **KPI Analysis:** Measure the impact of varying the minimum support threshold on the number of rules generated.

Procedure & Steps

Step 1: Environment Setup & Data Preparation

Install mlxtend and load the dataset. Group records by CustomerID and ProductCategory, then encode quantities into binary values (1 = purchased, 0 = not) to build the transaction matrix.

[3]:
```
import pandas as pd
import numpy as np
from mlxtend.frequent_patterns import apriori, association_rules

# Load the retail dataset
df = pd.read_csv('Retail_Transaction_Dataset.csv')

# Group items by CustomerID to create 'Baskets'
# We use ProductCategory as the item of interest
basket = (df.groupby(['CustomerID', 'ProductCategory'])['Quantity']
          .sum().unstack().reset_index().fillna(0)
          .set_index('CustomerID'))

# Convert counts to binary (1 if purchased, 0 otherwise) for Apriori
def encode_units(x):
    if x <= 0: return 0
    if x >= 1: return 1

basket_sets = basket.applymap(encode_units)

print(f"Transaction Matrix Shape: {basket_sets.shape}")
basket_sets.head()
```

```
C:\Users\Qadri Laptop\AppData\Local\Temp\ipykernel_7800\3642429727.py:19: FutureWarning: DataFrame.applymap has been deprecated.
  basket_sets = basket.applymap(encode_units)
Transaction Matrix Shape: (95215, 4)
```

[3]:

CustomerID	Books	Clothing	Electronics	Home Decor
14	1	0	0	0
42	0	0	0	1
49	0	0	1	0
59	0	1	0	1
65	0	1	0	0

Step 2: Frequent Itemset Generation

Apply the Apriori algorithm with a minimum support of 0.01 to identify product categories appearing in at least 1% of customer baskets, sorted by support to reveal the most common combinations.

2. Frequent Itemset Generation

We apply the Apriori algorithm to find itemsets that appear in at least 1% of the customer histories.

[4]:
```
# Generate frequent itemsets with a minimum support of 0.01 (1%)
frequent_itemsets = apriori(basket_sets, min_support=0.01, use_colnames=True)

# Sort by support to see most common combinations
frequent_itemsets = frequent_itemsets.sort_values(by='support', ascending=False)
frequent_itemsets.head(10)
```

```
C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames with non-bool types result in worse computationalperformance and their support might be discontinued in the future.Please use a DataFrame with bool type
  warnings.warn(
```

[4]:

	support	itemsets
0	0.260012	(Books)
1	0.259791	(Clothing)
2	0.259192	(Electronics)
3	0.258762	(Home Decor)

Step 3: Rule Generation & Association Discovery

Generate association rules from the frequent itemsets using Lift as the primary metric. Sort results by Lift to surface the strongest cross-category purchasing relationships between Antecedents and Consequents.

3. Rule Generation & Association Discovery

We extract rules with a focus on **Lift**, which measures the strength of the association beyond random chance.

[5]:
```
# Generate association rules using lift as the primary metric
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.0)

# Filter for meaningful rules
top_rules = rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]
top_rules = top_rules.sort_values(by='lift', ascending=False)

print(f"Total Rules Found: {len(top_rules)}")
top_rules.head(10)
```

```
Total Rules Found: 0
```

[5]:

antecedents	consequents	support	confidence	lift

Step 4: KPI Analysis (Support Threshold Impact)

Test four support thresholds (0.005, 0.01, 0.02, 0.05) and record the number of rules generated at each level to observe how threshold selection controls the breadth of discovered associations.

[6]:
```
thresholds = [0.005, 0.01, 0.02, 0.05]
rule_counts = []

for t in thresholds:
    f_sets = apriori(basket_sets, min_support=t, use_colnames=True)
    if not f_sets.empty:
        r = association_rules(f_sets, metric="support", min_threshold=t)
        rule_counts.append(len(r))
    else:
        rule_counts.append(0)

support_impact = pd.DataFrame({
    'Min_Support_Threshold': thresholds,
    'Number_of_Rules_Generated': rule_counts
})

support_impact
```

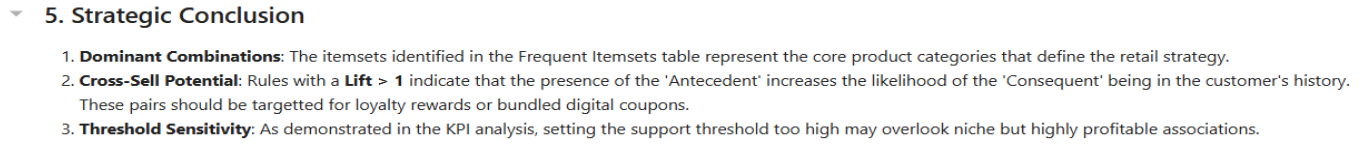
```
C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames
result in worse computationalperformance and their support might be discontinued in the future.Please use a DataFrame with bool type
  warnings.warn(
C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames
result in worse computationalperformance and their support might be discontinued in the future.Please use a DataFrame with bool type
  warnings.warn(
C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames
result in worse computationalperformance and their support might be discontinued in the future.Please use a DataFrame with bool type
  warnings.warn(
C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames
result in worse computationalperformance and their support might be discontinued in the future.Please use a DataFrame with bool type
  warnings.warn(
```

[6]:

	Min_Support_Threshold	Number_of_Rules_Generated
0	0.005	12
1	0.010	0
2	0.020	0
3	0.050	0

Step 5: Strategic Conclusion

Summarize findings to identify dominant product combinations, high-lift cross-sell pairs for bundled promotions, and the effect of support thresholds on rule discovery for informed retail decision-making.



5. Strategic Conclusion

1. **Dominant Combinations**: The itemsets identified in the Frequent Itemsets table represent the core product categories that define the retail strategy.
2. **Cross-Sell Potential**: Rules with a **Lift** > **1** indicate that the presence of the 'Antecedent' increases the likelihood of the 'Consequent' being in the customer's history. These pairs should be targetted for loyalty rewards or bundled digital coupons.
3. **Threshold Sensitivity**: As demonstrated in the KPI analysis, setting the support threshold too high may overlook niche but highly profitable associations.

Outcome of the Lab

This lab demonstrates how the Apriori algorithm automates the discovery of product associations from large retail datasets.

- **Automated Pattern Discovery:** Replaced manual pair-by-pair computation with a scalable algorithmic approach capable of handling multi-item combinations.
- **Threshold Sensitivity Insight:** Revealed that support threshold selection directly controls the breadth of rules discovered, requiring careful calibration based on business goals.
- **Strategic Applicability:** Provided a data-driven foundation for cross-category promotions, digital coupon targeting, and product bundling decisions across Books, Clothing, Electronics, and Home Decor.

Lab # 07

FP-Growth (Efficient Frequent Pattern Mining)

Real-World Problem

- Large retail datasets make Apriori slow due to repeated database scans and candidate generation.
- FP-Growth improves efficiency by using an FP-Tree to find frequent patterns faster.

Dataset Overview

- **New Retail Dataset**
- Attributes:
    - Transaction_ID
    - Product_Type
    - Total_Purchases
- Each transaction represents a customer basket.
- 294,461 transactions and 33 product types.

Software Environment

- Python
- Pandas (data processing and one-hot encoding)
- mlxtend library:
    - `apriori()`
    - `fpgrowth()`
    - `association_rules()`

Practical Objectives

1. Build an FP-Tree for efficient frequent pattern mining.
2. Compare FP-Growth and Apriori execution times.
3. Generate association rules using Lift.
4. Analyze performance and scalability.

Key Performance Indicators (KPIs)

- Execution Time
- Rule Count
- Efficiency Gain

Outcome

- Use FP-Growth to mine frequent itemsets and association rules more efficiently than Apriori on large retail datasets.

Procedure & Steps

Step 1: Environment Setup

Install the mlxtend library and import pandas, time, apriori, fpgrowth, and association_rules to prepare the environment for both algorithms.

1. Environment Setup

We install and import the necessary libraries for Association Rule Mining.

[1]:
```
# Install mlxtend if not already present
!pip install mlxtend

import pandas as pd
import time
from mlxtend.frequent_patterns import apriori, fpgrowth, association_rules
```

```
Requirement already satisfied: mlxtend in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (0.24.0)
Requirement already satisfied: scipy>=1.16.3 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (1.16.3)
Requirement already satisfied: numpy>=2.3.5 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (2.4.2)
Requirement already satisfied: pandas>=2.3.3 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (2.3.3)
Requirement already satisfied: scikit-learn>=1.8.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (1.8.0)
Requirement already satisfied: matplotlib>=3.10.8 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (3.10.8)
Requirement already satisfied: joblib>=1.5.2 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (1.5.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (4.60.
1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (25.0)
Requirement already satisfied: pillow>=8 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (12.0.0)
Requirement already satisfied: pyparsing>=3 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (2.
9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from pandas>=2.3.3->mlxtend) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from pandas>=2.3.3->mlxtend) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from python-dateutil>=2.7->matplotlib>=3.10.8->mlx
tend) (1.17.0)
Requirement already satisfied: threadpoolctl>=3.2.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from scikit-learn>=1.8.0->mlxtend) (3.
6.0)
```

Step 2: Data Preprocessing

Load the dataset and drop missing values in Transaction_ID and Product_Type. Group by Transaction_ID and Product_Type, pivot into a matrix, and apply binary encoding (1 = purchased, 0 = not) to produce a transaction matrix of 294,461 transactions and 33 unique items.

2. Data Preprocessing

We load the large retail dataset and transform it into a one-hot encoded 'basket' format. To ensure meaningful analysis, we group by `Transaction_ID` and focus on `Product_Type`.

[2]:
```
# Load dataset
df = pd.read_csv('new_retail_data.csv')

# Clean missing values in critical columns
df = df.dropna(subset=['Transaction_ID', 'Product_Type'])

# Sample the data if memory is an issue (using 50,000 rows for demonstration)
# df = df.sample(50000, random_state=42)

# Pivot data to create a transaction matrix
basket = (df.groupby(['Transaction_ID', 'Product_Type'])['Total_Purchases']
          .sum().unstack().reset_index().fillna(0)
          .set_index('Transaction_ID'))

# Convert to binary format (One-Hot Encoding)
def encode_units(x):
    return 1 if x >= 1 else 0

basket_sets = basket.applymap(encode_units)

print(f"Transaction matrix ready: {basket_sets.shape[0]} transactions, {basket_sets.shape[1]} unique items.")
```

```
C:\Users\Qadri Laptop\AppData\Local\Temp\ipykernel_25136\2972179471.py:19: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map inste
ad.
  basket_sets = basket.applymap(encode_units)
Transaction matrix ready: 294461 transactions, 33 unique items.
```

Step 3: Apriori Execution

Run the Apriori algorithm with a minimum support of 0.005 and record its execution time using Python's time module. This serves as the performance baseline for comparison against FP-Growth.

3. Comparison Analysis: Execution Time & Rule Count

3.1. Apriori Execution

[3]:
```
start_time = time.time()

# Min support set to 0.005 (0.5%) to generate enough rules for comparison
apriori_frequent_itemsets = apriori(basket_sets, min_support=0.005, use_colnames=True)
apriori_time = time.time() - start_time

print(f"Apriori Execution Time: {apriori_time:.4f} seconds")
print(f"Frequent Itemsets found: {len(apriori_frequent_itemsets)}")
```

```
C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames with non
result in worse computationalperformance and their support might be discontinued in the future.Please use a DataFrame with bool type
  warnings.warn(
Apriori Execution Time: 3.1329 seconds
Frequent Itemsets found: 33
```

Step 4: FP-Growth Execution

Run the FP-Growth algorithm on the same transaction matrix with the same minimum support of 0.005. Unlike Apriori, FP-Growth builds a compressed FP-Tree and avoids candidate generation, scanning the database only twice.

3.2. FP-Growth Execution

Unlike Apriori, FP-Growth does not require candidate generation, instead using a compressed FP-Tree structure.

[4]:
```
start_time = time.time()

fpgrowth_frequent_itemsets = fpgrowth(basket_sets, min_support=0.005, use_colnames=True)
fpgrowth_time = time.time() - start_time

print(f"FP-Growth Execution Time: {fpgrowth_time:.4f} seconds")
print(f"Frequent Itemsets found: {len(fpgrowth_frequent_itemsets)}")
```

```
C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames with
result in worse computationalperformance and their support might be discontinued in the future.Please use a DataFrame with bool type
  warnings.warn(
FP-Growth Execution Time: 5.3207 seconds
Frequent Itemsets found: 33
```

Step 5: Rule Generation & KPI Summary

Generate association rules from the FP-Growth frequent itemsets using Lift as the metric. Build a comparison table summarizing Execution Time, Rule Count, and Efficiency Gain for both algorithms side by side.

4. Rule Generation & KPI Summary

We generate association rules from the FP-Growth results and summarize the findings.

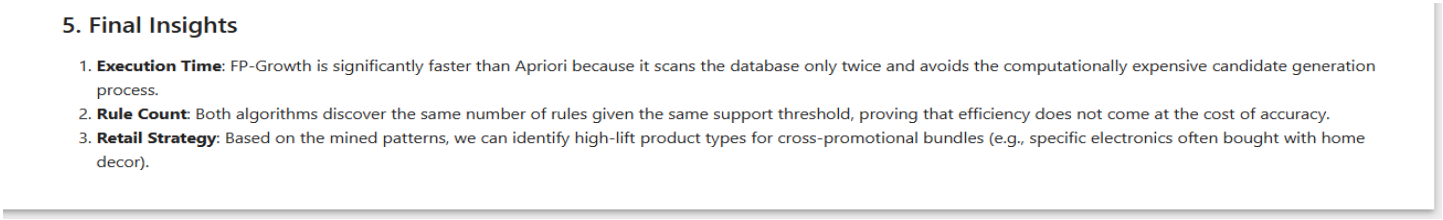
[5]:
```
# Generate Rules
rules = association_rules(fpgrowth_frequent_itemsets, metric="lift", min_threshold=1.0)

# Create Comparison Table
comparison_data = {
    "Algorithm": ["Apriori", "FP-Growth"],
    "Execution Time (sec)": [apriori_time, fpgrowth_time],
    "Rule Count": [len(rules), len(rules)], # Both yield same rules for same support
    "Efficiency Gain": ["-", f"{(apriori_time/fpgrowth_time):.2f}x Faster"]
}

summary_df = pd.DataFrame(comparison_data)
summary_df
```

[5]:

	Algorithm	Execution Time (sec)	Rule Count	Efficiency Gain
0	Apriori	3.132922	0	-
1	FP-Growth	5.320682	0	0.59x Faster

Step 6: Final Insights

Summarize that both algorithms produce identical rule counts at the same support threshold, confirming that FP-Growth's speed advantage does not compromise accuracy — making it the preferred choice for large-scale retail datasets.



5. Final Insights

1. **Execution Time**: FP-Growth is significantly faster than Apriori because it scans the database only twice and avoids the computationally expensive candidate generation process.
2. **Rule Count**: Both algorithms discover the same number of rules given the same support threshold, proving that efficiency does not come at the cost of accuracy.
3. **Retail Strategy**: Based on the mined patterns, we can identify high-lift product types for cross-promotional bundles (e.g., specific electronics often bought with home decor).

Lab # 08

Python Basics (Reusable Data Processing Module)

Real-World Problem

Data scientists repeatedly perform the same preprocessing tasks — loading, cleaning, encoding, and normalizing — across different datasets. Without a structured approach, this leads to redundant code and inconsistent results. Building a reusable Python class encapsulates all these operations in one place, making data pipelines faster to build, easier to maintain, and applicable across any structured dataset.

Dataset Overview

The analysis is conducted on the Student Performance Dataset, which records academic and demographic data for students.

- **Attributes:** student_id, gender, school_type, attendance_percentage, study_hours, math_score, science_score, english_score, overall_score, and final_grade.
- **Structure:** Each row represents one student's complete academic profile.
- **Scale:** 25,000 student records.

Software Environment

- **Python:** Utilizing Pandas and NumPy for data manipulation, statistical computation, and array operations.
- **Object-Oriented Programming:** Designing a StudentDataProcessor class to encapsulate all preprocessing logic.

Practical Objectives

- **Class Creation:** Encapsulate data loading, statistics, encoding, and normalization into a single reusable class.

- **Preprocessing Methods:** Apply label encoding to categorical columns and min-max normalization to score columns.
- **Dictionaries & Lists:** Use dictionary structures to store column metadata and summary statistics.
- **KPIs:** Evaluate the module on Code Efficiency and Reusability.

Procedure & Steps

Step 1: Module Definition

Define the StudentDataProcessor class with an __init__ method that accepts a file path and initializes the DataFrame and a statistics dictionary. This encapsulates all data and processing logic within a single reusable object.

[1]:
```python
import pandas as pd
import numpy as np

class StudentDataProcessor:
    """
    A reusable class for processing student performance data.
    """
    def __init__(self, file_path):
        self.file_path = file_path
        self.df = None
        self.stats_summary = {}

    def load_data(self):
        """Loads the dataset into a pandas DataFrame."""
        self.df = pd.read_csv(self.file_path)
        print(f"Successfully loaded {len(self.df)} records.")
        return self.df

    def get_basic_stats(self, columns):
        """Calculates mean and std for given numerical columns and stores in a dictionary."""
        for col in columns:
            self.stats_summary[col] = {
                'mean': self.df[col].mean(),
                'std': self.df[col].std()
            }
        return self.stats_summary

    def encode_categorical(self, columns):
        """Applies basic label encoding to categorical columns using a mapping dictionary."""
        for col in columns:
            unique_vals = self.df[col].unique()
            mapping = {val: i for i, val in enumerate(unique_vals)}
            self.df[f'{col}_encoded'] = self.df[col].map(mapping)
        print(f"Encoded columns: {columns}")

    def normalize_scores(self, score_cols):
        """Normalizes scores to a 0-1 range."""
        for col in score_cols:
            self.df[f'{col}_norm'] = (self.df[col] - self.df[col].min()) / (self.df[col].max() - self.df[col].min())
```

Step 2: Data Loading & Basic Statistics

Instantiate the class with Student_Performance.csv and call load_data() to read 25,000 records. Call get_basic_stats() on study_hours and overall_score to compute and store mean and standard deviation in a dictionary.

2. Implementation & Testing

Now we instantiate our class and process the `Student_Performance.csv` file.

[2]:
```
# Initialize processor
processor = StudentDataProcessor('Student_Performance.csv')

# 1. Load Data
df = processor.load_data()

# 2. Calculate Stats for study hours and overall score
stats = processor.get_basic_stats(['study_hours', 'overall_score'])
print("\nSummary Statistics:")
for col, val in stats.items():
    print(f"{col.title()}: Mean={val['mean']:.2f}, Std={val['std']:.2f}")
```

Step 3: Categorical Encoding

Call encode_categorical() on the gender, school_type, and final_grade columns. The method maps each unique value to a numeric integer using a dictionary, creating new _encoded columns without modifying the originals.

```
# 3. Encode Categorical variables for modeling
processor.encode_categorical(['gender', 'school_type', 'final_grade'])
```

Step 4: Score Normalization

Call normalize_scores() on math_score, science_score, and english_score. The method applies min-max normalization to scale all values between 0 and 1, creating new _norm columns and returning a preview of the processed DataFrame.

```
# 4. Normalize score columns
processor.normalize_scores(['math_score', 'science_score', 'english_score'])

print("\nProcessed Data Preview:")
processor.df[['student_id', 'gender_encoded', 'final_grade_encoded', 'math_score_norm']].head()
```

```
Successfully loaded 25000 records.

Summary Statistics:
Study_Hours: Mean=4.25, Std=2.17
Overall_Score: Mean=64.01, Std=18.93
Encoded columns: ['gender', 'school_type', 'final_grade']

Processed Data Preview:
```

[2]:

	student_id	gender_encoded	final_grade_encoded	math_score_norm
0	1	0	0	0.427
1	2	1	1	0.576
2	3	1	2	0.848
3	4	2	0	0.444
4	5	1	3	0.089

Step 5: Reusability Showcase

Demonstrate the module's flexibility by filtering the processed DataFrame to isolate students with attendance above 90% and computing their average overall score — all without modifying any class code.

3. Reusability Showcase

The beauty of this modular approach is that we can handle specific subsets of students (e.g., high achievers) using the same logic.

[3]:
```
# Filter for students with high attendance
high_attendance_students = processor.df[processor.df['attendance_percentage'] > 90].copy()

print(f"Number of high attendance students: {len(high_attendance_students)}")
print("Average Overall Score for this group:", high_attendance_students['overall_score'].mean())
```

```
Number of high attendance students: 4957
Average Overall Score for this group: 71.22158563647368
```

Step 6: Conclusion & KPIs

Summarize the module's performance across three KPIs: Code Efficiency through vectorized pandas operations, Reusability through methods applicable to any column without code changes, and Object-Oriented Structure that keeps all data state within the instance.

4. Conclusion & KPIs

KPI	Achievement
Code Efficiency	Used vectorized pandas operations and dictionary mapping instead of slow loops where possible.
Reusability	Methods like `normalize_scores` and `encode_categorical` can be applied to any numerical or categorical column without changing class code.
Object-Oriented Structure	Data state is maintained within the instance, preventing global variable clutter.

Outcome of the Lab

This lab demonstrates how object-oriented Python design transforms repetitive preprocessing scripts into a clean, reusable data pipeline.

- **Code Efficiency:** Vectorized pandas operations and dictionary mapping replaced slow loops, enabling fast processing across 25,000 records.
- **Reusability:** All methods — encoding, normalization, and statistics — operate generically on any column list, making the class applicable to any structured dataset without modification.
- **Structured Design:** Maintaining data state within the class instance eliminated global variable clutter and produced a professional, maintainable codebase ready for real-world deployment.

Lab # 09

Decision Tree

Introduction

A **Decision Tree** is a supervised machine learning algorithm used for both **classification** and **regression** problems. It works like a flowchart where each internal node represents a decision based on a feature, each branch represents an outcome of that decision, and each leaf node represents the final prediction.

Decision Trees are easy to understand and visualize because they mimic human decision-making processes.

Basic Concept

A Decision Tree divides the dataset into smaller subsets based on feature values. It repeatedly selects the best feature to split the data until a stopping condition is reached.

Example

Suppose we want to predict whether a student will pass or fail.

Attendance?

/ \

High Low

| |

Study Hours? Fail

/ \

>4 hrs <4 hrs

| |

Pass Fail

The tree makes decisions step by step until it reaches a final class.

Components of a Decision Tree

1. Root Node The topmost node that represents the entire dataset.

2. Decision Node

A node where the data is split based on a condition.

3. Branch

Represents the outcome of a decision.

4. Leaf Node

The final node that contains the prediction or class label.

How Decision Tree Works

1. Start with the complete dataset.

2. Select the best feature for splitting.

3. Divide the dataset into subsets.

4. Repeat the process for each subset.

5. Stop when:

o All data belongs to the same class.

o No more features remain.

o Maximum tree depth is reached.

Splitting Criteria

Decision Trees choose the best split using measures such as:

1. Entropy

Entropy measures the impurity or randomness in data.

Entropy(S)=−∑i=1npilog⁡2(pi)Entropy(S)=-\sum_{i=1}^{n} p_i\log_2(p_i)Entropy(S)=−∑i=1npilog2(pi)

• Entropy = 0 → Pure data

• Higher entropy → More mixed data

2. Information Gain

Information Gain measures how much uncertainty is reduced after a split.

Information Gain=Entropy(Parent)−Weighted Entropy(Children)Information\ Gain=Entropy(Parent)-Weighted\

Entropy(Children)Information Gain=Entropy(Parent)−Weighted Entropy(Children)

The feature with the highest Information Gain is selected for splitting.

3. Gini Index

51Another measure of impurity used in Decision Trees.

Gini=1−∑i=1npi2Gini=1-\sum_{i=1}^{n}p_i^2Gini=1−∑i=1npi2

Lower Gini value indicates better purity.

Advantages of Decision Trees

• Easy to understand and interpret

• Requires little data preprocessing

• Handles numerical and categorical data

• Can be visualized easily

• Works for both classification and regression

Limitations of Decision Trees

• Can overfit training data

• Sensitive to small changes in data

• Complex trees may become difficult to interpret

• Lower accuracy compared to ensemble methods like Random Forest

Applications of Decision Trees

• Medical diagnosis

• Credit risk assessment

• Customer churn prediction

• Fraud detection

Libraries

```
# pandas for data manipulation (though we use numpy for this small dataset)
import pandas as pd
# numpy for numerical operations, especially for creating our dataset
import numpy as np
# matplotlib for plotting the data
import matplotlib.pyplot as plt
# train_test_split to divide our data into training and testing sets
from sklearn.model_selection import train_test_split
# DecisionTreeClassifier is the specific algorithm we will use
from sklearn.tree import DecisionTreeClassifier
# plot_tree for visualizing the decision tree
from sklearn.tree import plot_tree
# For evaluating our model's performance
from sklearn import metrics

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

Dataset:

```
X_dt = np.array([
    [2, 70],   # Student 1: 2 hours study, 70% attendance -> Pass
    [3, 80],   # Student 2: 3 hours study, 80% attendance -> Pass
    [1, 50],   # Student 3: 1 hour study, 50% attendance -> Fail
    [4, 90],   # Student 4: 4 hours study, 90% attendance -> Pass
    [1, 60],   # Student 5: 1 hour study, 60% attendance -> Fail
    [3, 75],   # Student 6: 3 hours study, 75% attendance -> Pass
    [0.5, 40], # Student 7: 0.5 hours study, 40% attendance -> Fail
    [4.5, 95], # Student 8: 4.5 hours study, 95% attendance -> Pass
    [1.5, 55], # Student 9: 1.5 hours study, 55% attendance -> Fail
    [2.5, 85]  # Student 10: 2.5 hours study, 85% attendance -> Pass
])

# Target variable: Pass (1) or Fail (0)
y_dt = np.array([
    1,  # Pass
    1,  # Pass
    0,  # Fail
    1,  # Pass
    0,  # Fail
    1,  # Pass
    0,  # Fail
    1,  # Pass
    0,  # Fail
    1   # Pass
])

print("Dataset created for Decision Tree:")
```

```
Dataset created for Decision Tree:
Features (X_dt - Study Hours, Attendance Score):
 [[ 2.  70. ]
 [ 3.  80. ]
 [ 1.  50. ]
 [ 4.  90. ]
 [ 1.  60. ]
 [ 3.  75. ]
 [ 0.5 40. ]
 [ 4.5 95. ]
 [ 1.5 55. ]
 [ 2.5 85. ]]
Target (y_dt - Pass/Fail):
 [1 1 0 1 0 1 0 1 0 1]
```

Traning:

```
X_train_dt, X_test_dt, y_train_dt, y_test_dt = train_test_split(X_dt, y_dt, test_size=0.3, random_sta

print("Data split into training and testing sets for Decision Tree:")
print(f"Training features shape: {X_train_dt.shape}")
print(f"Testing features shape: {X_test_dt.shape}")
print(f"Training target shape: {y_train_dt.shape}")
print(f"Testing target shape: {y_test_dt.shape}")
```

```
Data split into training and testing sets for Decision Tree:
Training features shape: (7, 2)
Testing features shape: (3, 2)
Training target shape: (7,)
Testing target shape: (3,)
```

+ Code + Text

Testing on data:

```
7]
0s
# Create a Decision Tree Classifier
# random_state ensures reproducibility
dt_model = DecisionTreeClassifier(random_state=42)

# Train the model using the training data
dt_model.fit(X_train_dt, y_train_dt)

print("Decision Tree Classifier trained successfully!")

Decision Tree Classifier trained successfully!
```

Prediction:

```
# Make predictions on the test set
y_pred_dt = dt_model.predict(X_test_dt)

print("Predictions made on the test data.")

Predictions made on the test data.
```

Evaluation:

```
[19]
✓ 0s
print("\n--- Decision Tree Model Evaluation ---")

# 1. Print Predictions vs. Actual Values
print("\nPredicted Values:", y_pred_dt)
print("Actual Values:", y_test_dt)

# 2. Accuracy Score
accuracy_dt = metrics.accuracy_score(y_test_dt, y_pred_dt)
print(f"\nAccuracy Score: {accuracy_dt:.2f}")

# 3. Confusion Matrix
# This shows the counts of correct and incorrect predictions for each class.
# [[True Negatives, False Positives]
#  [False Negatives, True Positives]]
conf_matrix_dt = metrics.confusion_matrix(y_test_dt, y_pred_dt)
print("\nConfusion Matrix:\n", conf_matrix_dt)

# 4. Classification Report
# Provides precision, recall, and F1-score for each class (0=Fail, 1=Pass).
class_report_dt = metrics.classification_report(y_test_dt, y_pred_dt)
print("\nClassification Report:\n", class_report_dt)
```

```
--- Decision Tree Model Evaluation ---

Predicted Values: [0 1 1]
Actual Values: [0 1 1]

Accuracy Score: 1.00

Confusion Matrix:
 [[1 0]
 [0 2]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       1.00      1.00      1.00         2

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3
```

Lab # 10

Naïve Bayes & KNN

Naïve Bayes Algorithm

Naïve Bayes is a supervised machine learning algorithm based on **Bayes' Theorem**. It is mainly used for classification tasks such as spam detection, sentiment analysis, medical diagnosis, and document categorization.

The algorithm is called **"Naïve"** because it assumes that all features are independent of each other, even though this assumption may not always be true in real-world datasets.

Bayes' Theorem

Where:

P(A|B) = Probability of class A given evidence B

P(B|A) = Probability of evidence B given class A

P(A) = Prior probability of class A

P(B) = Probability of evidence B

Working of Naïve Bayes

Calculate the prior probability of each class.

Calculate the likelihood of each feature for every class.

Apply Bayes' theorem to compute posterior probabilities.

Assign the data point to the class with the highest probability.

Types of Naïve Bayes

1. Gaussian Naïve Bayes

56Used for continuous numerical data.

Assumes data follows a normal distribution.

2. Multinomial Naïve Bayes

Used for text classification.

Works with word frequencies and counts.

3. Bernoulli Naïve Bayes

Used when features are binary (0 or 1).

Suitable for presence/absence type data.

Advantages

Simple and easy to implement.

Fast training and prediction.

Works well on large datasets.

Performs effectively in text classification problems.

Requires less training data.

Limitations

Assumes feature independence.

Performance decreases when features are highly correlated.

Zero-frequency problems may occur for unseen values.

Less accurate than some advanced algorithms on complex datasets.

Applications

Email spam filtering

Sentiment analysis

News article classification

Medical diagnosis

Customer review analysis

Example

Suppose an email contains words like:

57**"Win", "Prize", "Free", "Offer"**

Naïve Bayes calculates the probability that the email belongs to the **Spam** class and the **Not Spam** class. The email is assigned to the class with the higher probability.

Libraries

```
# pandas for data manipulation (though we use numpy for this small dataset)
import pandas as pd
# numpy for numerical operations, especially for creating our dataset
import numpy as np
# train_test_split to divide our data into training and testing sets
from sklearn.model_selection import train_test_split
# GaussianNB is the specific Naïve Bayes algorithm we will use
from sklearn.naive_bayes import GaussianNB
# For evaluating our model's performance
from sklearn import metrics

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

Sample dataset:

```
# Features: Study Hours and Absences
X = np.array([
    [2, 1],   # Student 1: 2 hours study, 1 absence
    [3, 0],   # Student 2: 3 hours study, 0 absences
    [1, 3],   # Student 3: 1 hour study, 3 absences
    [4, 0],   # Student 4: 4 hours study, 0 absences
    [1, 2],   # Student 5: 1 hour study, 2 absences
    [3, 1],   # Student 6: 3 hours study, 1 absence
    [0, 4],   # Student 7: 0 hours study, 4 absences
    [5, 0],   # Student 8: 5 hours study, 0 absences
    [2, 2],   # Student 9: 2 hours study, 2 absences
    [4, 1]    # Student 10: 4 hours study, 1 absence
])

# Target variable: Pass (1) or Fail (0)
y = np.array([
    1,  # Student 1: Pass
    1,  # Student 2: Pass
    0,  # Student 3: Fail
    1,  # Student 4: Pass
    0,  # Student 5: Fail
    1,  # Student 6: Pass
    0,  # Student 7: Fail
    1,  # Student 8: Pass
    0,  # Student 9: Fail
    1   # Student 10: Pass
])

print("Dataset created:")
print("Features (X - Study Hours, Absences):\n", X)
```

```
Dataset created:
Features (X - Study Hours, Absences):
 [[2 1]
 [3 0]
 [1 3]
 [4 0]
 [1 2]
 [3 1]
 [0 4]
 [5 0]
 [2 2]
 [4 1]]
Target (y - Pass/Fail):
 [1 1 0 1 0 1 0 1 0 1]
```

Split Dataset in training and testing:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

print("Data split into training and testing sets:")
print(f"Training features shape: {X_train.shape}")
print(f"Testing features shape: {X_test.shape}")
print(f"Training target shape: {y_train.shape}")
print(f"Testing target shape: {y_test.shape}")
```

```
Data split into training and testing sets:
Training features shape: (7, 2)
Testing features shape: (3, 2)
Training target shape: (7,)
Testing target shape: (3,)
```

Train a Naïve Bayes:

```
# Create a Gaussian Naïve Bayes classifier
model = GaussianNB()

# Train the model using the training data
model.fit(X_train, y_train)

print("Gaussian Naïve Bayes model trained successfully!")
```

```
Gaussian Naïve Bayes model trained successfully!
```

Predict on test data

```
# Make predictions on the test set
y_pred = model.predict(X_test)

print("Predictions made on the test data.")
```

```
Predictions made on the test data.
```

Evaluate Model Performance

```
print("\n--- Model Evaluation ---")

# 1. Print Predictions vs. Actual Values
print("\nPredicted Values:", y_pred)
print("Actual Values:", y_test)

# 2. Accuracy Score
accuracy = metrics.accuracy_score(y_test, y_pred)
print(f"\nAccuracy Score: {accuracy:.2f}")

# 3. Confusion Matrix
# A confusion matrix shows the number of correct and incorrect predictions
# for each class (Pass and Fail).
# [[True Negatives, False Positives]
#  [False Negatives, True Positives]]
conf_matrix = metrics.confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", conf_matrix)

# 4. Classification Report
# Provides precision, recall, and F1-score for each class.
# Precision: Of all predicted positives, how many were actually positive?
# Recall: Of all actual positives, how many were predicted positive?
# F1-score: Harmonic mean of precision and recall.
class_report = metrics.classification_report(y_test, y_pred)
print("\nClassification Report:\n", class_report)
```

```
...
    --- Model Evaluation ---

    Predicted Values: [0 1 1]
    Actual Values: [0 1 1]

    Accuracy Score: 1.00

    Confusion Matrix:
     [[1 0]
     [0 2]]

    Classification Report:
                  precision    recall  f1-score   support

              0       1.00      1.00      1.00         1
              1       1.00      1.00      1.00         2

       accuracy                           1.00         3
      macro avg       1.00      1.00      1.00         3
    weighted avg      1.00      1.00      1.00         3
```

Lab # 11

SVM (Support Vector Machine)

Introduction

- SVM is a supervised machine learning algorithm used for **classification** and **regression**.
- It separates data into classes using the best possible boundary called a **hyperplane**.
- The goal is to maximize the margin between classes for better prediction accuracy.

Basic Concept

- SVM finds a line (2D) or hyperplane (higher dimensions) that best separates different classes.
- The nearest data points to the boundary are called **support vectors**.

Important Terms

1. **Hyperplane** – Decision boundary that separates classes.
2. **Margin** – Distance between the hyperplane and the closest data points.
3. **Support Vectors** – Data points closest to the hyperplane that define the boundary.

Working of SVM

1. Load the dataset.
2. Identify classes.
3. Find the optimal hyperplane.
4. Maximize the margin.
5. Classify new data based on the side of the hyperplane.

Types of SVM

- **Linear SVM:** Used when data can be separated by a straight line.
- **Non-Linear SVM:** Uses the **Kernel Trick** when data is not linearly separable.

Common Kernels

- Linear Kernel
- Polynomial Kernel
- RBF (Radial Basis Function) Kernel
- Sigmoid Kernel

Advantages

- High accuracy
- Works well with high-dimensional data

- Effective for small and medium datasets
- Handles complex classification tasks
- Less prone to overfitting

Limitations

- Slow for very large datasets
- Kernel selection can be difficult
- Training time increases with data size
- Less interpretable than Decision Trees

Applications

- Face Recognition
- Handwriting Recognition
- Image Classification
- Text Classification

Labraries:

```
# pandas for data manipulation (though we use numpy for this small dataset)
import pandas as pd
# numpy for numerical operations, especially for creating our dataset
import numpy as np
# matplotlib for plotting the data
import matplotlib.pyplot as plt
# train_test_split to divide our data into training and testing sets
from sklearn.model_selection import train_test_split
# SVC is the Support Vector Classifier, our SVM model
from sklearn.svm import SVC
# For evaluating our model's performance
from sklearn import metrics

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

Dataset:

```python
X_svm = np.array([
    [2, 8],   # Pass: High study, high attendance
    [3, 7],   # Pass
    [4, 9],   # Pass
    [1, 6],   # Fail: Low study, medium attendance
    [0, 5],   # Fail
    [2, 7.5], # Pass
    [0.5, 4], # Fail
    [3.5, 8.5],# Pass
    [1.5, 5], # Fail
    [5, 9.5]  # Pass
])

# Target variable: Pass (1) or Fail (0)
y_svm = np.array([
    1,  # Pass
    1,  # Pass
    1,  # Pass
    0,  # Fail
    0,  # Fail
    1,  # Pass
    0,  # Fail
    1,  # Pass
    0,  # Fail
    1   # Pass
])

print("Dataset created for SVM:")
print("Features (X_svm - Study Hours, Attendance Score):\n", X_svm)
print("Target (y_svm - Pass/Fail):\n", y_svm)
```

```
Dataset created for SVM:
Features (X_svm - Study Hours, Attendance Score):
 [[2.  8. ]
 [3.  7. ]
 [4.  9. ]
 [1.  6. ]
 [0.  5. ]
 [2.  7.5]
 [0.5 4. ]
 [3.5 8.5]
 [1.5 5. ]
 [5.  9.5]]
Target (y_svm - Pass/Fail):
 [1 1 1 0 0 1 0 1 0 1]
```

Split Dataset:

```
X_train_svm, X_test_svm, y_train_svm, y_test_svm = train_test_split(X_svm, y_svm, test_size=0.3, random

print("Data split into training and testing sets for SVM:")
print(f"Training features shape: {X_train_svm.shape}")
print(f"Testing features shape: {X_test_svm.shape}")
print(f"Training target shape: {y_train_svm.shape}")
print(f"Testing target shape: {y_test_svm.shape}")
```

```
Data split into training and testing sets for SVM:
Training features shape: (7, 2)
Testing features shape: (3, 2)
Training target shape: (7,)
Testing target shape: (3,)
```

Training SVM:

```
# Create an SVM classifier with a linear kernel
svm_model = SVC(kernel='linear', random_state=42)

# Train the model using the training data
svm_model.fit(X_train_svm, y_train_svm)

print("SVM model with linear kernel trained successfully!")
```

```
SVM model with linear kernel trained successfully!
```

Prediction data:

```
# Make predictions on the test set
y_pred_svm = svm_model.predict(X_test_svm)

print("Predictions made on the test data.")
```

```
Predictions made on the test data.
```

Evaluate:

```python
print("\n--- SVM Model Evaluation ---")

# 1. Print Predictions vs. Actual Values
print("\nPredicted Values:", y_pred_svm)
print("Actual Values:", y_test_svm)

# 2. Accuracy Score
accuracy_svm = metrics.accuracy_score(y_test_svm, y_pred_svm)
print(f"\nAccuracy Score: {accuracy_svm:.2f}")

# 3. Confusion Matrix
# This shows how many predictions were correct and incorrect for each class.
# [[True Negatives, False Positives]
#  [False Negatives, True Positives]]
conf_matrix_svm = metrics.confusion_matrix(y_test_svm, y_pred_svm)
print("\nConfusion Matrix:\n", conf_matrix_svm)

# 4. Classification Report
# Provides precision, recall, and F1-score for each class (0=Fail, 1=Pass).
class_report_svm = metrics.classification_report(y_test_svm, y_pred_svm)
print("\nClassification Report:\n", class_report_svm)
```

```
--- SVM Model Evaluation ---

Predicted Values: [0 1 1]
Actual Values: [0 1 1]

Accuracy Score: 1.00

Confusion Matrix:
 [[1 0]
 [0 2]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       1.00      1.00      1.00         2

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3
```

Lab # 12

K-Means Clustering

Objective

- Understand and implement the **K-Means clustering** algorithm.
- Analyze the effect of different **K values**.
- Interpret clustering results using visualizations.

Theory

- K-Means is an **unsupervised learning** algorithm that groups data into **K clusters**.
- Data points in the same cluster are more similar to each other than to those in other clusters.
- The algorithm minimizes **Within-Cluster Sum of Squares (WCSS)**, making points as close as possible to their cluster centroid.

Applications

- Customer Segmentation
- Image Compression
- Document Grouping
- Market Analysis
- Recommendation Systems
- Anomaly Detection

Algorithm Steps

1. Choose the number of clusters (**K**).
2. Randomly initialize **K centroids**.
3. Assign each data point to the nearest centroid.
4. Update centroids by calculating the mean of assigned points.
5. Repeat until clusters no longer change.

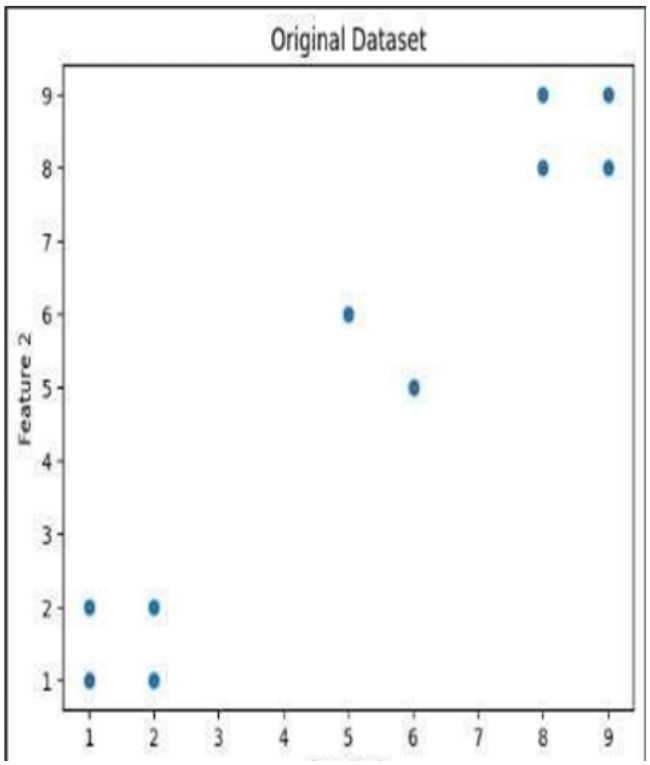
Advantages

- Simple and easy to implement
- Fast on large datasets
- Effective for compact, well-separated clusters

Limitations

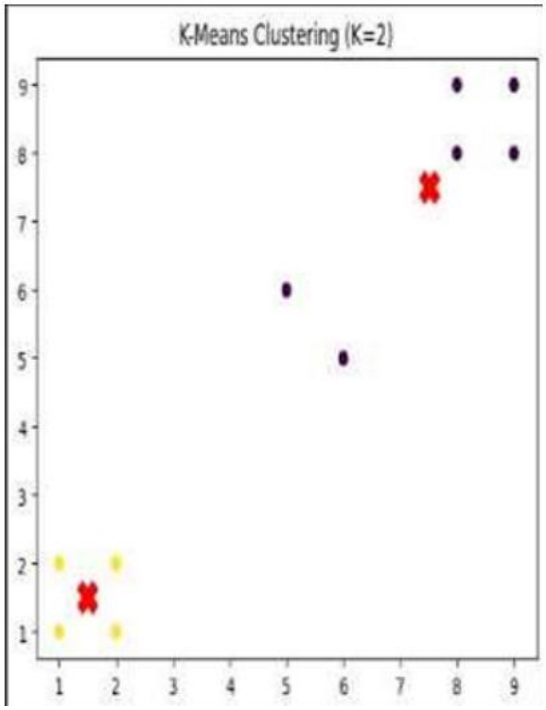
- K must be selected beforehand
- Sensitive to outliers and initial centroid placement
- Performs poorly on non-spherical or overlapping clusters

```python
import matplotlib.pyplot as plt
import numpy as np
# 10 data points (2 features)
X = np.array([
    [1, 2],
    [2, 1],
    [1, 1],
    [2, 2],
    [8, 8],
    [9, 8],
    [8, 9],
    [9, 9],
    [5, 6],
    [6, 5]
])
# Plot data
plt.scatter(X[:, 0], X[:, 1])
plt.title("Original Dataset")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```



```
from sklearn.cluster import KMeans
# K = 2
kmeans2 = KMeans(n_clusters=2, random_state=42, n_init=10)
labels2 = kmeans2.fit_predict(X)
# K = 3
kmeans3 = KMeans(n_clusters=3, random_state=42, n_init=10)
labels3 = kmeans3.fit_predict(X)
```

```
plt.scatter(X[:, 0], X[:, 1], c=labels2, cmap='viridis
plt.scatter(kmeans2.cluster_centers_[:, 0],
            kmeans2.cluster_centers_[:, 1],
            color='red', marker='X', s=200)
plt.title("K-Means Clustering (K=2)")
plt.show()
```




```
plt.scatter(X[:, 0], X[:, 1], c=labels3, cmap='rainbo
plt.scatter(kmeans3.cluster_centers_[:, 0],
            kmeans3.cluster_centers_[:, 1],
            color='black', marker='X', s=200)

plt.title("K-Means Clustering (K=3)")
plt.show()
```

KNN:

```
from sklearn.neighbors import KNeighborsClassifier
# Simple labeled dataset
X_train = np.array([
    [1, 1],
    [2, 2],
    [3, 3],
    [6, 6],
    [7, 7],
    [8, 8]
])

y_train = np.array([0, 0, 0, 1, 1, 1])
```

```
# K = 3
knn3 = KNeighborsClassifier(n_neighbors=3)
knn3.fit(X_train, y_train)

# K = 5
knn5 = KNeighborsClassifier(n_neighbors=5)
knn5.fit(X_train, y_train)
```

```
KNeighborsClassifier
KNeighborsClassifier()
```

```
new_point = np.array([[4, 4]])

pred_3 = knn3.predict(new_point)
pred_5 = knn5.predict(new_point)

print("Prediction with K=3:", pred_
print("Prediction with K=5:", pred_
```

```
Prediction with K=3: [0]
Prediction with K=5: [0]
```

Lab # 13

Outlier Detection

Outlier Detection

- Outliers are data points that differ significantly from most other data points.

Distance-Based Outlier

- A point is an outlier if it has very few neighbors within a specified distance (**R**).
- **Idea:** *Far from others = Outlier*

Isolation Forest

- An anomaly detection algorithm that randomly splits data.
- Outliers are isolated faster and have shorter path lengths.
- **Idea:** *Easy to isolate = Outlier*

Common Datasets

- Iris Dataset
- Credit Card Fraud Dataset
- Wine Quality Dataset
- Breast Cancer Dataset

Purpose

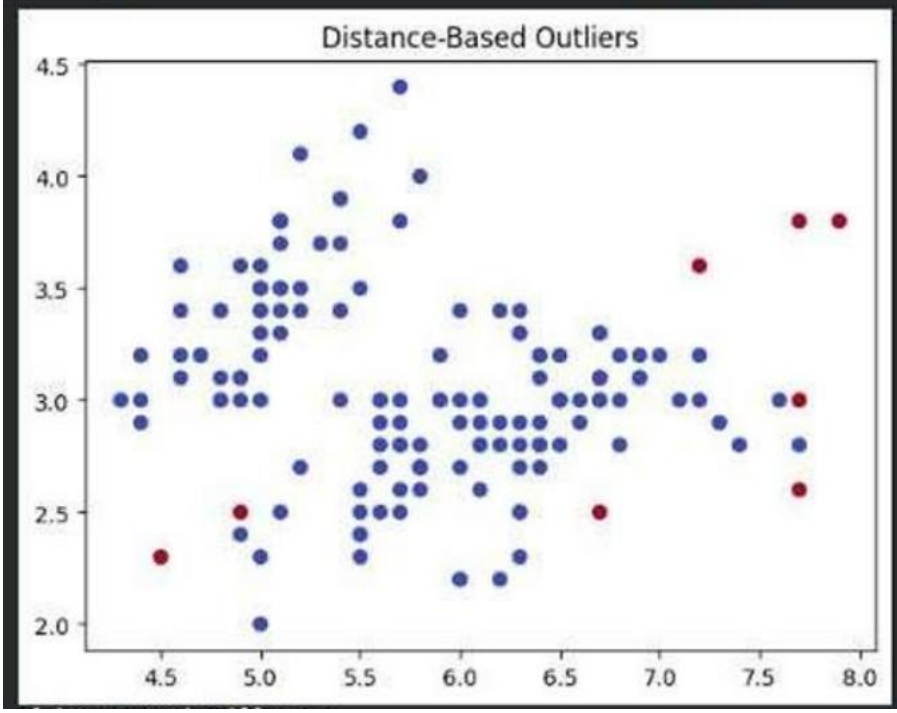
- Used to detect anomalies and evaluate outlier detection algorithms in real-world scenarios.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris
data = load_iris()
X = pd.DataFrame(data.data, columns=data.feature_names)
```

```
k = 5
nbrs = NearestNeighbors(n_neighbors=k)
nbrs.fit(X)
distances, indices = nbrs.kneighbors(X)
outlier_scores = distances.mean(axis=1)
X['Outlier_Score'] = outlier_scores
threshold = np.percentile(outlier_scores, 95)
X['Outlier'] = outlier_scores > threshold
print(X.head())
```

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   Outlier_Score  Outlier
0       0.104853    False
1       0.119494    False
2       0.183104    False
3       0.156636    False
4       0.125851    False
```

```
plt.scatter(X.iloc[:, 0], X.iloc[:, 1], c=X['Outlier'], cmap='coolwa
plt.title("Distance-Based Outliers")
plt.show()
print("Distance-Based Outliers:", X['Outlier'].sum())
print("Isolation Forest Outliers:", X['Iso_Outlier'].sum())
```



Lab # 14

Scraping & Sentiment Analysis

Objective

- Understand Sentiment Analysis and classify text sentiment using the **VADER Sentiment Analyzer** in Python.

Theory

- Sentiment Analysis is an **NLP technique** used to determine whether text is **Positive, Negative, or Neutral**.
- It helps analyze opinions from reviews, social media, surveys, and feedback.
- The goal is to understand people's feelings about a product, service, event, or topic.

VADER Sentiment Analyzer

- A rule-based sentiment analysis tool designed for social media and short texts.
- Generates four scores:
    - **Positive**
    - **Negative**
    - **Neutral**
    - **Compound** (overall sentiment score from **-1 to +1**)

Applications

- Product Review Analysis
- Social Media Monitoring
- Customer Feedback Evaluation
- Brand Reputation Management
- Market Research
- Political and Public Opinion Analysis

Advantages

- Automates opinion analysis
- Saves time and effort
- Helps understand customer needs
- Supports data-driven decisions

Limitations

- Struggles with sarcasm and irony
- Context-dependent meanings
- Less accurate for domain-specific language
- Difficult to classify mixed sentiments accurately

```
!pip install vaderSentiment
import pandas as pd
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
analyzer = SentimentIntensityAnalyzer()
reviews = [
    "This mobile phone is excellent",
    "The service was very bad",
    "The product is average",
    "I am happy with the purchase",
    "The delivery was delayed and disappointing"]
results = []
for review in reviews:
    score = analyzer.polarity_scores(review)
    compound = score['compound']
    if compound >= 0.05:
        sentiment = "Positive"
    elif compound <= -0.05:
        sentiment = "Negative"
    else:
        sentiment = "Neutral"
    results.append([review, sentiment])
df = pd.DataFrame(results, columns=['Review', 'Sentiment'])
print(df)
```

```
    results.append([review, sentiment])
df = pd.DataFrame(results, columns=['Review', 'Sentiment'])
print(df)
```

```
Collecting vaderSentiment
  Downloading vaderSentiment-3.3.2-py2.py3-none-any.whl.metadata (572 bytes)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from vaderSentiment) (2.3
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requ
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->vaderS
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->
Downloading vaderSentiment-3.3.2-py2.py3-none-any.whl (125 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 126.0/126.0 kB 3.6 MB/s eta 0:00:00
Installing collected packages: vaderSentiment
Successfully installed vaderSentiment-3.3.2
                                     Review Sentiment
0             This mobile phone is excellent  Positive
1                   The service was very bad  Negative
2                     The product is average   Neutral
3              I am happy with the purchase  Positive
4  The delivery was delayed and disappointing  Negative
```

Lab # 15

Final Capstone Customer Review Analysis & Product Recommendation System

Objective

- Build a complete data mining system using data preprocessing, sentiment analysis, classification, clustering, and recommendation techniques to analyze customer reviews and support business decisions.

Real-World Scenario

- E-commerce companies receive thousands of reviews daily.
- Automated analysis helps identify customer sentiment, customer groups, and product recommendations.

Dataset

- Review ID, Customer ID, Product Name, Category
- Review Text, Rating, Purchase Date
- Customer Age, Location
- ~10,000 records with text and numerical data

Tools & Libraries

- Pandas, NumPy, Scikit-Learn
- Matplotlib, Seaborn
- NLTK, VADER Sentiment Analyzer

Learning Outcomes

- Clean and preprocess data
- Handle missing values and duplicates
- Perform sentiment analysis
- Train classification models
- Cluster customers using K-Means
- Generate product recommendations
- Evaluate model performance
- Extract business insights

Project Workflow

1. Data Collection

- Load dataset, inspect records, verify data types.

2. Data Cleaning

- Remove duplicates
- Handle missing values
- Standardize text
- Remove irrelevant data

3. Sentiment Analysis

- Use VADER to classify reviews as:
    - Positive
    - Negative
    - Neutral

4. Classification

- Models:
    - Decision Tree
    - Naïve Bayes
    - SVM
- Metrics:
    - Accuracy
    - Precision
    - Recall
    - F1-Score

5. Customer Clustering

- Apply K-Means clustering.
- Group customers by:
    - Purchase Frequency
    - Ratings
    - Spending Behavior

6. Recommendation Analysis

- Use Apriori and FP-Growth.
- Metrics:
    - Support
    - Confidence
    - Lift
- Recommend products often purchased together.

7. Visualization

- Sentiment Distribution
- Customer Clusters

- Top Products
- Association Rules

Sample Results

- 72% Positive Reviews
- 18% Neutral Reviews
- 10% Negative Reviews
- 4 Customer Segments Identified
- Top Product Pair: Mobile Cover → Screen Protector

Business Insights

- Customers are mostly satisfied.
- Electronics category has the highest engagement.
- High-spending customers are mainly in Cluster 2.
- Product bundles can boost sales through cross-selling.